

All Bookmarks

1. Simple MVC	Simple MVC
2. WebApp2	Dependency Injection
3. Console App	CRUD
4. WebApp3	CRUD
5. WebApp4	EF Core Code First
6. WebApp5	EF Core DB First
7. WebApi	Web Api using EF Core
8. WebApi6	State Management
9. Angular+React	Client-side Development
10. WebApp7	Authentication & Authorization
11. Hosting	

18. WebApp1

1. ASP.NET Core MVC (Model, View, Controller)

- **Model:** Represents the data and business logic of the application. It contains classes, data objects, and rules.
 - **View:** Responsible for displaying data to the user. Usually written in Razor syntax (.cshtml).
 - **Controller:** Handles user requests, interacts with the Model, and returns a View or data.
 - **Flow:** User → Controller → Model → View → User.
 - **Purpose:** Separates concerns, making applications easier to maintain and test.
-

2. Razor Page

- Introduced in ASP.NET Core 2.0 for page-focused development.
 - Combines HTML markup with C# code using Razor syntax (@).
 - Each page has a .cshtml file (markup) and a .cshtml.cs file (page model for logic).
 - Ideal for simple page-based applications without the full MVC structure.
-

3. Validation Attribute

- Special attributes in the **System.ComponentModel.DataAnnotations** namespace used to validate model properties.
 - Examples:
 - [Required] → Ensures a value is provided.
 - [StringLength(50)] → Limits string length.
 - [Range(1, 100)] → Restricts numerical values.
 - [EmailAddress] → Validates email format.
 - Helps in both **server-side** and **client-side** validation.
-

4. Tag Helpers

- Server-side components in ASP.NET Core that make Razor views more HTML-like.
 - Allow you to work with HTML elements using C# attributes.
 - Examples:
 - <form asp-action="Create">
 - <input asp-for="Name" />
 - <label asp-for="Email"></label>
 - Improve maintainability and reduce repetitive code.
-

5. Server-Side Validation

- Validation done on the server after the form is submitted.
- Ensures security because client-side scripts can be bypassed.
- Uses model validation with Data Annotations and `ModelState.IsValid` in controllers.
- Often paired with client-side validation for better user experience.

6. Action Method

- A method inside a Controller that responds to HTTP requests.
- Returns data (View, JSON, file, redirect, etc.) depending on the request.
- Examples:
 - `public IActionResult Index() { return View(); }`
 - `public IActionResult Details(int id) { ... }`
- Can have attributes like `[HttpGet]`, `[HttpPost]`, `[Authorize]` to control behavior.

Topic	Description	Key Points / Examples
ASP.NET Core MVC (Model, View, Controller)	Architectural pattern for separating concerns in web apps.	Model: Data + business logic. View: UI using Razor (.cshtml). Controller: Handles requests, calls model, returns view. Flow: User → Controller → Model → View → User.
Razor Page	Page-focused approach introduced in ASP.NET Core 2.0.	.cshtml for markup + .cshtml.cs for page logic. Uses Razor syntax @. Simpler than full MVC for page-based apps.
Validation Attribute	Data annotation attributes to validate model properties.	<code>[Required]</code> , <code>[StringLength]</code> , <code>[Range]</code> , <code>[EmailAddress]</code> . Defined in <code>System.ComponentModel.DataAnnotations</code> . Works for both client and server-side validation.
Tag Helpers	Server-side features to generate HTML dynamically in Razor.	<code><form asp-action="Create"><input asp-for="Name" /><label asp-for="Email"></label></code> . Improves maintainability and reduces manual HTML coding.
Server-Side Validation	Validation executed on the server after form submission.	Uses <code>Data Annotations</code> + <code>ModelState.IsValid</code> . More secure than client-side validation. Prevents bypassing by disabling JS.
Action Method	Method in a Controller that processes HTTP requests.	Example: <code>public IActionResult Index() { return View(); }</code> . Can return Views, JSON, Redirects. Can be decorated with <code>[HttpGet]</code> , <code>[HttpPost]</code> , <code>[Authorize]</code> .

MyRazorPage.cshtml

```
@{
    ViewData["Title"] = "My Razor Page";
    string name = ViewBag.Name;
    int rollNo = ViewBag.RollNo;
    int table_of = ViewBag.TableOf;
    DateTime now = ViewBag.DateTime;
}

<h2>@ViewData["Title"]</h2>
<span>by @name (Roll: @rollNo)</span>
<p>Current Date: @now.ToString("dd MMM yyyy");
    Current Time: @now.ToString("hh : mm tt")</p>
<h4>Multiplication Table of @table_of</h4>
<ul style="list-style:none">
@for (int i=1; i<=10; i++)
{
    <li>@table_of x @i = @(table_of * i)</li>
}
</ul>
```

WebApp1_By_Sushanka Home Privacy MyRazorPage Create Student Record

My Razor Page

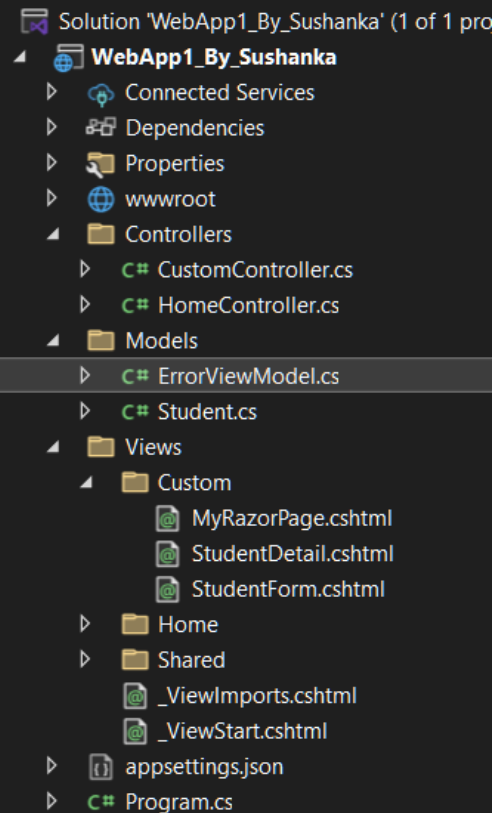
by Sushanka (Roll: 84)

Current Date: 10 Aug 2025; Current Time: 03 : 53 PM

Multiplication Table of 85

85 x 1 = 85
85 x 2 = 170
85 x 3 = 255
85 x 4 = 340
85 x 5 = 425
85 x 6 = 510
85 x 7 = 595
85 x 8 = 680
85 x 9 = 765
85 x 10 = 850

Search Solution Explorer (Ctrl+;)



Student.cs

```
using System.ComponentModel.DataAnnotations;

namespace WebApp1_By_Sushanka.Models
{
    public class Student
    {
        [Key]
        public int StdId{ get; set; }

        [Required(ErrorMessage = "Don't you have name?")]
        [MaxLength(20, ErrorMessage = "Are you writing essay?")]
        public string Name{ get; set; } = string.Empty;

        [Required(ErrorMessage = "Don't you exist?")]
        [StringLength(100)]
        public string Address { get; set; } = string.Empty;

        //[Required(ErrorMessage = "*Required")]

        [RegularExpression("^((Science|Management|Humanities|Education)$",
            ErrorMessage = "Faculty must be Science, Management,
Humanities or Education")]
        public string Faculty { get; set; } = "";

        [Range(16,70, ErrorMessage = "Take Social Security Allowance
and chill.")]
        public int Age { get; set; }

        [EmailAddress(ErrorMessage = "Invalid Email Address")]
        public string Email { get; set; } = "";
    }
}
```

StudentForm.cshtml

```
@model WebApp1_By_Sushanka.Models.Student

@{
    ViewData["Title"] = "StudentForm";
}

@* <h1>@((string)ViewData["Title"])</h1> *@
<h1>@ViewData["Title"]</h1>
@* <h1>@((string)ViewData["Title"])</h1>
<h1>@ViewData["title"].ToString()</h1> *@
<hr />

<form asp-action="FormSubmission" method="post">
    <div class="mb-3">
        <label asp-for="StdId">Student Id: </label>
        <input asp-for="StdId" />
        <span asp-validation-for="StdId" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Name">Name: </label>
        <input asp-for="Name" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Address">Address: </label>
        <input asp-for="Address" />
        <span asp-validation-for="Address" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Faculty">Faculty: </label>
        <input asp-for="Faculty" />
        <span asp-validation-for="Faculty" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Age">Age: </label>
        <input asp-for="Age" />
        <span asp-validation-for="Age" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Email">Email Address: </label>
        <input asp-for="Email" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>

    <button class="btn btn-primary">Create</button>
</form>

@* client side validation using unobtrusive validation *@
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

StudentForm

Student Id: The StdId field is required.

Name: Don't you have name?

Address: Don't you exist?

Faculty: The Faculty field is required.

Age: The Age field is required.

Email Address: The Email field is required.

Create

StudentForm

Student Id:

Name:

Address:

Faculty:

Age:

Email Address:

Create

StudentDetail.cshtml

```
@model WebApp1_By_Sushanka.Models.Student // importing Student Model
as Model(default)
@{
    ViewData["Title"] = "StudentDetail";
}

<h1>Student Details</h1>

<ul>
    @foreach (var property in Model.GetType().GetProperties()) @*
    looping through all the properties of Student Model*@
    {
        <li>@property.Name: @property.GetValue(Model)</li>
    }
</ul>

@* object sent from View() can be retrieved using Model *@
```

Student Details

- StdId: 10
- Name: Sushanka
- Address: Nepal
- Faculty: Science
- Age: 29
- Email: test@gmail.com

CustomController.cs

```
using Microsoft.AspNetCore.Mvc;
using Newtonsoft.Json;
using WebApp1_By_Sushanka.Models;

namespace WebApp1_By_Sushanka.Controllers
{
    public class CustomController : Controller
    {
        public IActionResult MyRazorPage()
        {
            ViewBag.Name = "Sushanka";
            ViewBag.RollNo = 84;
            ViewBag.TableOf = ViewBag.RollNo + 1;
            ViewBag.Datetime = DateTime.Now;
            return View();
        }

        //[HttpGet] no need as "GET" by default
        public IActionResult StudentForm()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken] // protects form Cross-Site
Request Forgery(CSRF) attacks
        public IActionResult FormSubmission(Student std)
        {
            if (ModelState.IsValid)
            {
                TempData["Student"] =
                JsonConvert.SerializeObject(std); // Pass data between requests,
                e.g., after RedirectToAction
                return RedirectToAction("StudentDetail"); // sendig
                data via get request expose the sensitive data
            }
            return View();
        }

        //[HttpPost]
        public IActionResult StudentDetail()
        {
            //var student_JSON = TempData["Student"] as string;
            will kepp data for one request only???
            var student_JSON = TempData.Peek("Student") as string;
            // TempData.Keep("Student") will preserve the value only for the next
            request after it's accessed.
            Student? student =
            JsonConvert.DeserializeObject<Student>(student_JSON);

            //ViewBag.student = student; // viewbag can be used also
            return View("StudentDetail", student);
        }
    }
}
```


19. WebApp2

Dependency Injection

Topic	Dependency Injection (DI)
Description	A design pattern where an object's dependencies are provided externally rather than created inside the object. Purpose: Increases flexibility, testability, and maintainability. In ASP.NET Core: Built-in DI container is used. Types: Constructor Injection, Method Injection, Property Injection. Service Lifetimes: • Singleton – one instance for the entire app. • Scoped – one instance per HTTP request. • Transient – new instance each time requested. Example: <pre>csharp
services.AddScoped<IService, ServiceImpl>();
</pre> Controller receives it: <pre>csharp
public MyController(IService service) { ... }
</pre>
Key Points / Examples	

Type	Description	Key Points / Examples
Constructor Injection	Dependencies are passed into a class through its constructor.	Most common in ASP.NET Core. Ensures dependency is available when object is created. Example: <pre>csharp
public class MyController {
 private readonly IService _service;
 public MyController(IService service) {
 _service = service; }
}
</pre>
Action Injection	Dependencies are passed directly as parameters to an action method.	Useful for occasional dependencies. No need to store dependency in a field. Example: <pre>csharp
public IActionResult Index([FromServices] IService service) { ... }
</pre>
Property Injection	Dependencies are assigned to public properties of a class.	Less common; dependency can be set after object creation. Requires public settable property. Example: <pre>csharp
public IService Service {
 get; set; }
</pre>

HomeController.cs

(Without DI)

- Controllers
 - DI_ActionController.cs
 - DI_ConstructorController.cs
 - DI_PropertyController.cs
 - HomeController.cs
- Models
 - ErrorViewModel.cs
 - IStudentRepository.cs
 - Student.cs
 - StudentRepository.cs
- Views
 - Home
 - Index.cshtml
 - Privacy.cshtml
 - Shared
 - _ViewImports.cshtml
 - _ViewStart.cshtml
- appsettings.json
- Program.cs

```
using System.Diagnostics;
using Microsoft.AspNetCore.Mvc;
using WebApp2_By_Sushanka.Models;

namespace WebApp2_By_Sushanka.Controllers
{
    public class HomeController : Controller
    {
        // without dependency injection
        public JsonResult Index()
        {
            StudentRepository _studentRepository
= new ();
            List<Student> allStudentDetails =
            _studentRepository.GetAllStudent();
            return Json(allStudentDetails);
        }

        public JsonResult GetStudentDetails(int
id)
        {
            StudentRepository _studentRepository
= new ();
            Student studentDetails =
            _studentRepository.GetStudentById(id);
            return Json(studentDetails);
        }
    }
}
```

```
[{"studentId":101,"name":"James","branch":"CSE","section":"A","gender":"Male"},
{"studentId":102,"name":"Smith","branch":"ETC","section":"B","gender":"Male"},
{"studentId":103,"name":"David","branch":"CSE","section":"A","gender":"Male"},
{"studentId":104,"name":"Sara","branch":"CSE","section":"A","gender":"Female"},
{"studentId":105,"name":"Pam","branch":"ETC","section":"B","gender":"Female"}]
```

With Dependency Injection

Student.cs

```
namespace WebApp2_By_Sushanka.Models
{
    public class Student
    {
        public int StudentId { get; set; }
        public string? Name { get; set; }
        public string? Branch { get; set; }
        public string? Section { get; set; }
        public string? Gender { get; set; }
    }
}
```

Models/IStudentRepository.cs

```
namespace WebApp2_By_Sushanka.Models
{
    public interface IStudentRepository
    {
        List<Student> GetAllStudent();
        Student GetStudentById(int StudentId);
    }
}
```

Models/StudentRepository.cs.cs

```
using System.Diagnostics.CodeAnalysis;

namespace WebApp2_By_Sushanka.Models
{
    public class StudentRepository : IStudentRepository
    {
        // all students data list
        public List<Student> DataSource ()
        {
            return new List<Student>()           // can be declared as []
            {
                new Student() { StudentId = 101, Name = "James", Branch = "CSE", Section = "A", Gender = "Male" },
                new () { StudentId = 102, Name = "Smith", Branch = "ETC", Section = "B", Gender = "Male" },
                new () { StudentId = 103, Name = "David", Branch = "CSE", Section = "A", Gender = "Male" },
                new () { StudentId = 104, Name = "Sara", Branch = "CSE", Section = "A", Gender = "Female" },
                new () { StudentId = 105, Name = "Pam", Branch = "ETC", Section = "B", Gender = "Female" }
            };
        }

        public Student GetStudentById(int id)
        {
            return DataSource().FirstOrDefault(std => std.StudentId == id)
            ?? new Student();
        }

        public List<Student> GetAllStudent()
        {
            return DataSource();
        }
    }
}
```

Controllers/DI_Constructor.cs

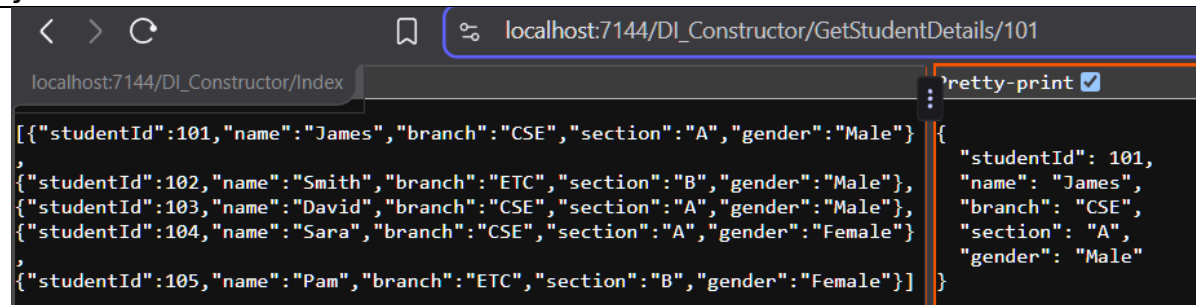
```
using Microsoft.AspNetCore.Mvc;
using WebApp2_By_Sushanka.Models;

namespace WebApp2_By_Sushanka.Controllers
{
    public class DI_ConstructorController : Controller
    {
        public readonly IStudentRepository _studentRepository;

        // Constructor Injection (DI)
        public DI_ConstructorController(IStudentRepository studentRepository)
        {
            _studentRepository = studentRepository;
        }

        public JsonResult Index()
        {
            List<Student> allStudentDetails =
            _studentRepository.GetAllStudent();
            return Json(allStudentDetails);
        }

        public JsonResult GetStudentDetails (int id)
        {
            Student studentDetails = _studentRepository.GetStudentById(id);
            return Json(studentDetails);
        }
    }
}
```



Controllers/DI_Action.cs

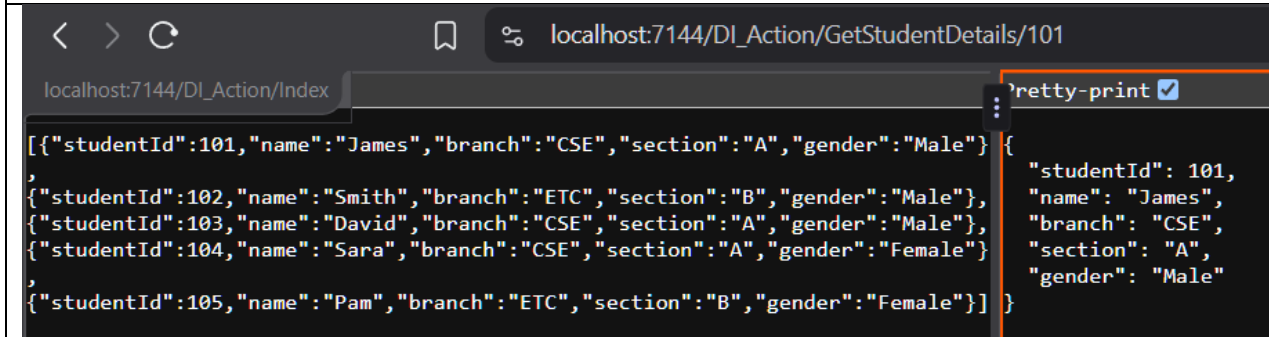
```
using Microsoft.AspNetCore.Mvc;
using WebApp2_By_Sushanka.Models;

namespace WebApp2_By_Sushanka.Controllers
{
    public class DI_ActionController : Controller
    {
        // instance must be of interface for loose coupling
        public JsonResult Index([FromServices] IStudentRepository _studentRepository)
        {
            List<Student> allStudentDetails =
            _studentRepository.GetAllStudent();
        }
    }
}
```

```

        return Json(allStudentDetails);
    }
    public JsonResult GetStudentDetails(int id, [FromServices]
IStudentRepository _studentRepository)
    {
        Student studentDetails = _studentRepository.GetStudentById(id);
        return Json(studentDetails);
    }
}

```



Controllers/DI_Property.cs

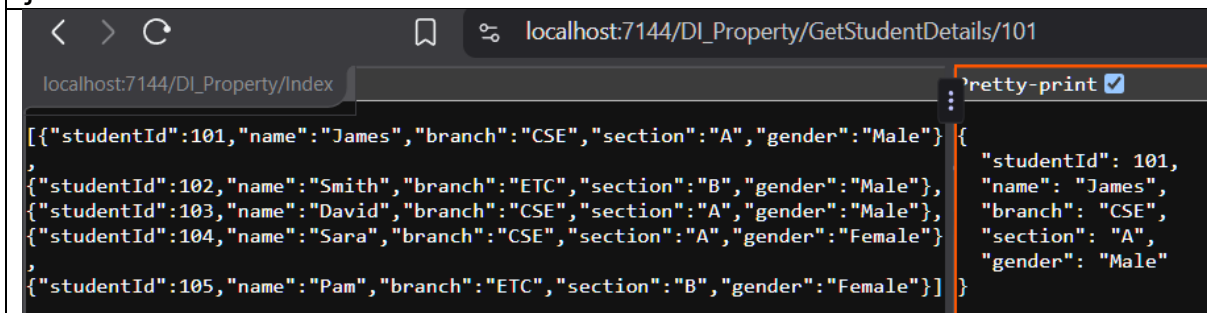
```

using Microsoft.AspNetCore.Mvc;
using WebApp2_By_Sushanka.Models;

namespace WebApp2_By_Sushanka.Controllers
{
    public class DI_PropertyController : Controller
    {
        [FromServices]
        public IStudentRepository _studentRepository { get; set; }
        public JsonResult Index()
        {
            var allStudentDetails = _studentRepository.GetAllStudent();
            return Json(allStudentDetails);
        }

        public JsonResult GetStudentDetails (int id)
        {
            var studentDetails = _studentRepository.GetStudentById(id);
            return Json(studentDetails);
        }
    }
}

```



Program.cs

```
using WebApp2_By_Sushanka.Models;
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();
// register the interface and implementation
// scoped:- new instance per Http request
builder.Services.AddScoped<IStudentRepository, StudentRepository>();
var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this
    for production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

20. ConsoleApp

Console Application in C#

- A simple program that runs in a command-line interface (CLI) without a graphical user interface (GUI).
 - Used for learning, utilities, or small tools.
 - Entry point is the `Main()` method inside a class.
 - Input/output via `Console.ReadLine()` and `Console.WriteLine()`.
 - Lightweight and fast to develop and run.
-

CRUD Operation

- Stands for **Create, Read, Update, Delete** — basic operations for managing data.
- Commonly used with databases or data storage.
- **Create**: Add new data (e.g., INSERT in SQL).
- **Read**: Retrieve data (e.g., SELECT).
- **Update**: Modify existing data (e.g., UPDATE).
- **Delete**: Remove data (e.g., DELETE).
- Essential in building applications that manage persistent data.

CRUD.cs

```
using Microsoft.Data.SqlClient;
using System;
using System.Data;

namespace ConsoleApp_Sushanka
{
    class CRUD
    {
        static readonly string connectionString = "Data source= localhost;
Initial Catalog=PlayerDB; Integrated Security = True;" +
        "Encrypt=True; TrustServerCertificate=True";

        static void Main()
        {
            while (true)
            {
                Console.WriteLine("\n----- Console App for CRUD
Operation -----");
                Console.Write($"1. Insert Player \n2. Read Players \n3.
Update Player \n4. Delete Player \n5. Exit \nChoose an option: ");
                var option = Convert.ToInt32(Console.ReadLine());

                switch (option)
                {
                    case 1:
                        InsertPlayer();
                        break;
```

```

        case 2:
            ReadPlayers();
            break;
        case 3:
            UpdatePlayer();
            break;
        case 4:
            DeletePlayer();
            break;
        case 5:
            Console.WriteLine("Exiting Program...");
            Environment.Exit(0);
            break;
        default:
            Console.WriteLine("Invalid option!!!");
            break;
    }
}

static void InsertPlayer()
{
    Console.Write("Enter Player Name: ");
    string name = Console.ReadLine();
    Console.Write("Enter Country : ");
    string? country = Console.ReadLine();
    Console.Write("Enter Club: ");
    string? club = Console.ReadLine();
    Console.Write("Enter Playing Postition: ");
    string? position = Console.ReadLine();
    Console.Write("Enter Preferred Foot : ");
    string foot = Console.ReadLine();

    // connecting to sb and executing query
    using SqlConnection conn = new(connectionString);
    conn.Open();
    string query = "insert into Players (Name, Country, Club,
Position, Foot) values(@name, @country, @club, @position, @foot)";
    SqlCommand cmd = new(query, conn);
    cmd.Parameters.AddWithValue("@name", name);
    cmd.Parameters.AddWithValue("@country", country);
    cmd.Parameters.AddWithValue("@club", club);
    cmd.Parameters.AddWithValue("@position", position);
    cmd.Parameters.AddWithValue("@foot", foot);
    cmd.ExecuteNonQuery();
    Console.WriteLine("Player inserted successfully!");
}

static void ReadPlayers()
{
    using SqlConnection conn = new(connectionString);
    conn.Open();
    string query = "select * from players";
    SqlCommand cmd = new(query, conn);
    SqlDataReader reader = cmd.ExecuteReader();

```



```

        //Console.WriteLine($"ID: \tName: \tCountry: \tClub:
\tPosition: \tFoot:");
        Console.WriteLine("{0,-3} {1,-10} {2,-10} {3,-10} {4,-10} {5,-
10}", "ID:", "Name:", "Country:", "Club:", "Position:", "Foot:");
        while (reader.Read())
        {
            //Console.WriteLine($"ID: {reader["Id"]} Name:
{reader["name"]} Country: {reader["country"]} Club: {reader["club"]}
Position: {reader["position"]}Foot: {reader["foot"]}");
            //Console.WriteLine($"{reader["Id"]} {reader["name"]}
{reader["country"]} {reader["club"]} {reader["position"]}
{reader["foot"]}");
            Console.WriteLine("{0,-3} {1,-10} {2,-10} {3,-10} {4,-10}
{5,-10}", reader["Id"], reader["Name"],
                reader["Country"], reader["Club"], reader["Position"],
reader["Foot"]);
        }
    }
    static void UpdatePlayer()
    {
        //Console.Write("Enter details to update ohterwise leave
blank.");
        Console.Write("Enter Player Id to update: ");
        int id= Convert.ToInt32(Console.ReadLine());
        Console.Write("Enter New Name: ");
        string name = Console.ReadLine();
        Console.Write("Enter New Country : ");
        string? country = Console.ReadLine();
        Console.Write("Enter New Club: ");
        string? club = Console.ReadLine();
        Console.Write("Enter New Playing Postition: ");
        string? position = Console.ReadLine();
        Console.Write("Enter New Preferred Foot : ");
        string foot = Console.ReadLine();

        using SqlConnection conn = new(connectionString);
        conn.Open();
        string query = "update Players set Name=@name,
Country=@country, Club=@club, Position=@position, Foot=@foot where
Id=@id";

        SqlCommand cmd = new(query, conn);
        cmd.Parameters.AddWithValue("@id", id);
        cmd.Parameters.AddWithValue("@name", name);
        cmd.Parameters.AddWithValue("@country", country);
        cmd.Parameters.AddWithValue("@club", club);
        cmd.Parameters.AddWithValue("@position", position);
        cmd.Parameters.AddWithValue("@foot", foot);
        int rows = cmd.ExecuteNonQuery();
        if (rows > 0)
            Console.WriteLine("Player updated successfully!");
        else
            Console.WriteLine("Player not found!");
    }
    static void DeletePlayer()
    {
        Console.Write("Enter Player Id to delete: ");

```

```

        int id = Convert.ToInt32(Console.ReadLine());

        using SqlConnection conn = new(connectionString);
        conn.Open();
        string query = "delete from Players where Id=@id";
        SqlCommand cmd = new(query, conn);
        cmd.Parameters.AddWithValue("@id", id);
        int rows = cmd.ExecuteNonQuery();
        if (rows > 0)
            Console.WriteLine("Player deleted successfully!");
        else
            Console.WriteLine("Player not found!");
    }
}

```

HP.PlayerDB - dbo.Players   SQLQuery1.sql...B (HP\me (58))

	Column Name	Data Type	Allow Nulls
	Id	int	☐
	Name	nchar(10)	☐
	Country	nchar(10)	☒
	Club	nchar(10)	☒
	Position	nchar(10)	☒
	Foot	nchar(10)	☐
			☐

 Results

 Messages

	Id	Name	Country	Club	Position	Foot
1	1	Ram	Nepal	Army	ST	Right
2	2	Messi	Argentina	Barca	RW	Left
3	3	Pedri	Spain	Barca	CM	Right
4	4	Hitler	Germany	Nazi	ST	Both

----- Console App for CRUD Operation -----

1. Insert Player
2. Read Players
3. Update Player
4. Delete Player
5. Exit

Choose an option: 1

Enter Player Name: Maradona

Enter Country : Argentina

Enter Club: Napoli

Enter Playing Postition: AM

Enter Preferred Foot : Left

Player inserted successfully!

----- Console App for CRUD Operation -----

1. Insert Player
2. Read Players
3. Update Player
4. Delete Player
5. Exit

Choose an option: 2

ID:	Name:	Country:	Club:	Position:	Foot:
1	Ram	Nepal	Army	ST	Right
2	Messi	Argentina	Barca	RW	Left
3	Pedri	Spain	Barca	CM	Right
4	Hitler	Germany	Nazi	ST	Both
5	Maradona	Argentina	Napoli	AM	Left

----- Console App for CRUD Operation -----

1. Insert Player
2. Read Players
3. Update Player
4. Delete Player
5. Exit

Choose an option: 3

Enter Player Id to update: 5

Enter New Name: Diego Jota

Enter New Country : Portugal

Enter New Club: Liverpool

Enter New Playing Postition: FW

Enter New Preferred Foot : Right

Player updated successfully!

----- Console App for CRUD Operation -----

1. Insert Player
2. Read Players
3. Update Player
4. Delete Player
5. Exit

Choose an option: 2

ID:	Name:	Country:	Club:	Position:	Foot:
1	Ram	Nepal	Army	ST	Right
2	Messi	Argentina	Barca	RW	Left
3	Pedri	Spain	Barca	CM	Right
4	Hitler	Germany	Nazi	ST	Both
5	Diego Jota	Portugal	Liverpool	FW	Right

----- Console App for CRUD Operation -----

1. Insert Player
2. Read Players
3. Update Player
4. Delete Player
5. Exit

Choose an option: 4

Enter Player Id to delete: 5

Player deleted successfully!

----- Console App for CRUD Operation -----

1. Insert Player
2. Read Players
3. Update Player
4. Delete Player
5. Exit

Choose an option: 5

Exiting Program...

C:\Users\me\source\repos\labs\ConsoleApp_Sushanka\ConsoleApp_Susha

Press any key to close this window . . .

21. WebApp3

CRUD Application:

- Stands for **Create, Read, Update, Delete** — basic operations for managing data.
- Common in web apps to manipulate database records.

ASP.NET Core MVC Template:

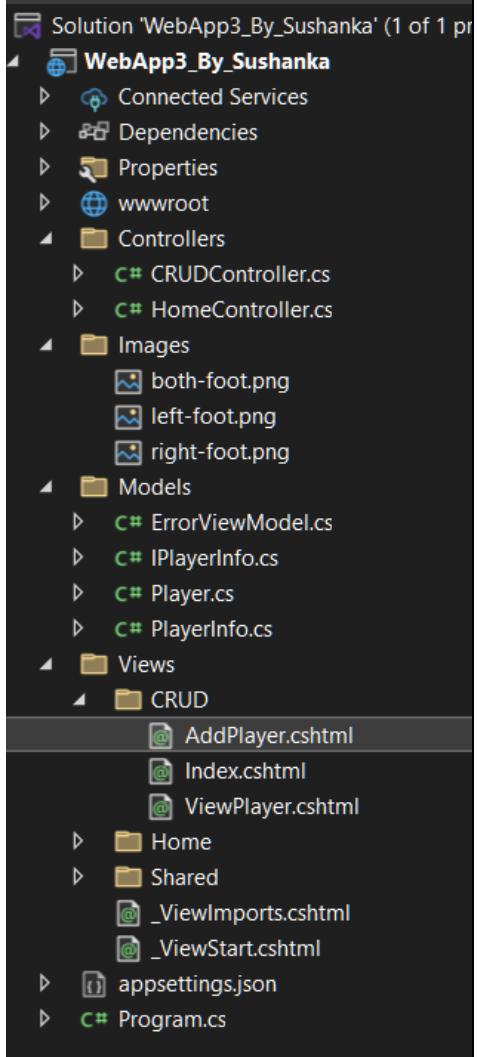
- Provides a starting point with Model-View-Controller architecture.
- Helps separate UI (View), business logic (Controller), and data (Model).
- Supports Razor views for dynamic HTML generation.

ADO.NET:

- A low-level data access technology in .NET for communicating with databases.
- Uses classes like `SqlConnection`, `SqlCommand`, `SqlDataReader` to execute SQL queries.
- Requires manual SQL query writing and connection handling.

Integration Overview:

- Use **ASP.NET Core MVC** to structure app and handle HTTP requests.
- Use **ADO.NET** inside Controllers to perform CRUD SQL operations.
- Use Models to represent data entities.
- Use Views to display forms and data lists.
- Store database connection string in `appsettings.json`.
- Perform database operations with parameterized queries to avoid SQL injection

Models/Player.cs <pre>using System.ComponentModel.DataAnnotations; namespace WebApp3_By_Sushanka.Models { public class Player { [Key] public int Id { get; set; } [Required(ErrorMessage = "*required")] public string Name { get; set; } = ""; public string? Country { get; set; } public string? Club { get; set; } public string? Position { get; set; } [Required(ErrorMessage = "*required")] public string Foot { get; set; } = ""; } }</pre>	
Models/IPlayerInfo.cs <pre>namespace WebApp3_By_Sushanka.Models { public interface IPlayerInfo { List<Player> GetAllPlayers(); Player GetPlayerById(int id); public void AddPlayer(Player p); public void UpdatePlayer(Player p); public void DeletePlayer(int id); } }</pre>	

Models/PlayerInfo.cs

```
using System.Configuration;
using Microsoft.Data.SqlClient;

namespace WebApp3_By_Sushanka.Models
{
    public class PlayerInfo : IPlayerInfo
    {
        readonly static string connectionString = "Data
source=localhost; Initial Catalog =Player_WebDb; " +
            "Integrated Security = True; Encrypt=True;
TrustServerCertificate=True";

        public List<Player> GetAllPlayers()
        {
            string query = "select * from players";
            using SqlConnection conn = new(connectionString);
            SqlCommand cmd = new(query, conn);
            conn.Open();

            SqlDataReader reader = cmd.ExecuteReader();

            List<Player> players = [];
            while (reader.Read())
            {
                players.Add(new Player()
                {
                    Id = Convert.ToInt32(reader["id"]),
                    Name = reader["Name"].ToString(),
                    Country = reader["Country"].ToString(),
                    Club = reader["Club"].ToString(),
                    Position = reader["Position"].ToString(),
                    Foot = reader["Foot"].ToString()
                });
            }
            return players;
        }

        public Player GetPlayerById(int id)
        {
            string query = "select * from players where Id=@id";
            using SqlConnection conn = new(connectionString);
            SqlCommand cmd = new(query, conn);
            cmd.Parameters.AddWithValue("@id", id);
            conn.Open();

            SqlDataReader reader = cmd.ExecuteReader();
            if (reader.Read())
```

```

        {
            Player player = new()
            {
                Id = Convert.ToInt32(reader["id"]),
                Name = reader["Name"].ToString(),
                Country = reader["Country"].ToString(),
                Club = reader["Club"].ToString(),
                Position = reader["Position"].ToString(),
                Foot = reader["Foot"].ToString()
            };
            return player;
        }
        return null;
    }

    public void AddPlayer(Player p)
    {
        string query = "insert into players values(@name,
@country, @club, @position, @foot)";
        using SqlConnection conn = new(connectionString);
        SqlCommand cmd = new(query, conn);
        conn.Open();
        cmd.Parameters.AddWithValue("@name", p.Name);
        cmd.Parameters.AddWithValue("@country", p.Country != null
? p.Country : DBNull.Value);
        cmd.Parameters.AddWithValue("@club", p.Club != null ?
p.Club : DBNull.Value);
        cmd.Parameters.AddWithValue("@position", p.Position !=
null ? p.Position : DBNull.Value);
        cmd.Parameters.AddWithValue("@foot", p.Foot);
        int rows = cmd.ExecuteNonQuery();
        //Console.WriteLine($"Affected rows: {rows}");
    }

    public void UpdatePlayer(Player p)
    {
        string query = "update Players set Name=@name,
Country=@country, Club=@club, Position=@position, Foot=@foot where
Id=@id";
        using SqlConnection conn = new(connectionString);
        SqlCommand cmd = new(query, conn);
        conn.Open();
        cmd.Parameters.AddWithValue("@id", p.Id);
        cmd.Parameters.AddWithValue("@name", p.Name);
        cmd.Parameters.AddWithValue("@country", p.Country != null
? p.Country : DBNull.Value);
        cmd.Parameters.AddWithValue("@club", p.Club != null ?
p.Club : DBNull.Value);
    }

```

```

        cmd.Parameters.AddWithValue("@position", p.Position !=
null ? p.Position : DBNull.Value);
        cmd.Parameters.AddWithValue("@foot", p.Foot);
        int rows = cmd.ExecuteNonQuery();
        //Console.WriteLine($"Affected rows: {rows}");
    }

    public void DeletePlayer(int id)
    {
        string query = "delete from players where Id=@id";
        using SqlConnection conn = new(connectionString);
        SqlCommand cmd = new(query, conn);
        cmd.Parameters.AddWithValue("@id", id);
        conn.Open();

        int rows = cmd.ExecuteNonQuery();
        //Console.WriteLine($"Affected rows: {rows}");
    }
}

```

Controllers/CRUDController.cs

```

using Microsoft.AspNetCore.Mvc;
using WebApp3_By_Sushanka.Models;

namespace WebApp3_By_Sushanka.Controllers
{
    public class CRUDController(IPlayerInfo player) : Controller    //
    constructor injection using primary constructor
    {
        private readonly IPlayerInfo _player = player;

        public IActionResult Index()
        {
            var players = _player.GetAllPlayers();
            return View(players);
        }

        public IActionResult ViewPlayer(int id)
        {
            var player = _player.GetPlayerById(id);
            return View(player);
        }

        [HttpGet]
        public IActionResult Addplayer()
        {
            ViewBag.Title = "Add Player";
            ViewBag.Action = "AddPlayer";
            ViewBag.Button = "Add";
        }
    }
}

```



```

        return View();
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult AddPlayer(Player p)
    {
        if (ModelState.IsValid)
        {
            _player.AddPlayer(p);
            TempData["SuccessMessage"] = "Player Added
Successfully";
            return RedirectToAction("Index");
        }
        return Content("Failed to register player.");
    }

    [HttpGet]
    public IActionResult EditPlayer(int id)
    {
        var player = _player.GetPlayerById(id);
        if (player == null) return Content("No player found");

        ViewBag.Title = "Edit Player";
        ViewBag.Action = "EditPlayer";
        ViewBag.Button = "Update";
        return View(viewName: "Views/CRUD/AddPlayer.cshtml",
model:player);
    }

    [HttpPost]
    [ValidateAntiForgeryToken]
    public IActionResult EditPlayer(Player p)
    {
        if (ModelState.IsValid)
        {
            _player.UpdatePlayer(p);

            TempData["SuccessMessage"] = "Player Updated
Successfully";
            return RedirectToAction("Index");
        }
        return Content("Failed to update player");
    }

    public IActionResult DeletePlayer(int id)
    {
        _player.DeletePlayer(id);

        TempData["SuccessMessage"] = "Player deleted successfully!";
        return RedirectToAction("Index");
    }
}

```

Views/CRUD/Index.cshtml

```
@model List<WebApp3_By_Sushanka.Models.Player>;

@{
    ViewData["Title"] = "CRUD";
}

@if (TempData["SuccessMessage"] != null)
{
    @* <div class="alert alert-success" id="flashMessage"> *@
    <div class="alert alert-success alert-dismissible fade show"
    role="alert" id="flashMessage">
        @TempData["SuccessMessage"]
        <button type="button" class="btn-close" data-bs-
    dismiss="alert" aria-label="Close"></button>
    </div>
}

<h1>Players</h1>

<a asp-action="AddPlayer">Create</a>
<table class="table table-striped table-bordered">
    <thead>
        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Foot</th>
            <th>Action</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var player in Model)
        {
            <tr>
                <td>@player.Id</td>
                <td>@player.Name</td>

                <td>
                    
                </td>
                <td><a asp-action="ViewPlayer" asp-route-
                id="@player.Id">View</a>
                    <a asp-action="EditPlayer" asp-route-
                id="@player.Id">Edit</a>
                    <a asp-action="DeletePlayer" asp-route-
                id="@player.Id">Delete</a>
                </td>
            </tr>
        }
    </tbody>
</table>
```

```

@* for flash message *@
<script>
    setTimeout(function () {
        var flash = document.getElementById('flashMessage');
        if (flash) flash.style.display = 'none';
    }, 3000); // 3 seconds
</script>

```

Views/CRUD/AddPlayer.cshtml

No need of EditPlayer.html (merged on AddPlayer)

```

@model WebApp3_By_Sushanka.Models.Player

@{
    // ViewData["Title"] = "AddPlayer";
}

<div class="container mt-4">
    <div class="row justify-content-center">
        <div class="col-md-4">
            @* <h3>AddPlayer</h3> *@
            <h3>@ViewBag.Title</h3>

            <form asp-controller="CRUD" asp-action="@ViewBag.Action"
method="post" asp-antiforgery="true">
                <div class="mb-3">
                    <label asp-for="Name" class="form-label">Player
Name:</label>

                    <input asp-for="Name" class="form-control"/>
                    <span asp-validation-for="Name" class="text-
danger"></span>
                </div>
                <div class="mb-3">
                    <label asp-for="Country" class="form-
label">Country:</label>

                    <input asp-for="Country" class="form-control"/>
                </div>
                <div class="mb-3">
                    <label asp-for="Club" class="form-label">Club:</label>
                    <input asp-for="Club" class="form-control" />
                </div>
                <div class="mb-3">
                    <label asp-for="Position" class="form-
label">Position:</label>

                    <input asp-for="Position" class="form-control" />
                </div>
                <div class="mb-3">
                    <label asp-for="Foot" class="form-label">Preferred
Foot:</label>

                    <select asp-for="Foot" class="form-select">
                        <option value="">--select--</option>
                        <option value="left">Left Foot</option>
                        <option value="right">Right Foot</option>
                        <option value="both">Both Foot</option>
                    </select>
                </div>
            </form>
        </div>
    </div>
</div>

```

```

        <span asp-validation-for="Foot" class="text-
danger"></span>
    </div>

    <button class="btn btn-primary">@ViewBag.Button</button>
</form>
</div>
</div>
</div>

@* client side validationo using unobtrusive validation *@
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

AddPlayer

Player Name:

*required

Country:

Club:

Position:

Preferred Foot:

*required

Add

AddPlayer

Player Name:

Country:

Club:

Position:

Preferred Foot:

Add

Edit Player

Player Name:

Country:

Club:

Position:

Preferred Foot:

Update

Player Updated Successfully



Players

[Create](#)

Id	Name	Foot	Action
1	Messi	⚽	View Edit Delete
2	Pogba	⚽	View Edit Delete
3	Garcia	⚽	View Edit Delete
4	D.Jota	⚽	View Edit Delete

Views/CRUD/ViewPlayer.cshtml

```
@{
    ViewData["Title"] = "ViewPlayer";
}

<h3>ViewPlayer</h3>

<a asp-action="AddPlayer">Create</a>
<table class="table table-striped table-bordered">
    <thead>
        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Country</th>
            <th>Club</th>
            <th>Position</th>
            <th>Foot</th>
            <th>Action</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>@Model.Id</td>
            <td>@Model.Name</td>
            <td>@Model.Country</td>
            <td>@Model.Club</td>
            <td>@Model.Position</td>
            <td>@Model.Foot</td>
            <td>
                <a asp-action="ViewPlayer" asp-route-id="@Model.Id">View</a>
                <a asp-action="EditPlayer" asp-route-id="@Model.Id">Edit</a>
                <a asp-action="DeletePlayer" asp-route-id="@Model.Id">Delete</a>
            </td>
        </tr>
    </tbody>
</table>
```

ViewPlayer

[Create](#)

Id	Name	Country	Club	Position	Foot	Action
1	Messi	Argentina	Barca	RW	left	View Edit Delete

An unhandled exception occurred while processing the request.

InvalidOperationException: Unable to resolve service for type 'WebApp3_By_Sushanka.Models.IPlayerInfo' while attempting to activate 'WebApp3_By_Sushanka.Controllers.CRUDController'.

Microsoft.Extensions.DependencyInjection.ActivatorUtilities.ThrowHelperUnableToResolveService(Type type, Type requiredBy)

Need to register to builder container

```
using WebApp3_By_Sushanka.Models;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

builder.Services.AddScoped<IPlayerInfo, PlayerInfo>();

var app = builder.Build();
```

22. EF Core Code First

Models/Player.cs

```
using System.ComponentModel.DataAnnotations;

namespace WebApp4_By_Sushanka.Models
{
    public class Player
    {
        [Key]
        public int Id { get; set; }
        [Required(ErrorMessage = "*required")]
        public string Name { get; set; } = "";
        public string? Country { get; set; }
        public string? Club { get; set; }
        public string? Position { get; set; }

        [Required(ErrorMessage = "*required")]
        public string Foot { get; set; } = "";
    }
}
```

Data/PlayerDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using WebApp4_By_Sushanka.Models;

namespace WebApp4_By_Sushanka.Data
{
    public class PlayerDbContext(DbContextOptions<PlayerDbContext> options) :
    DbContext(options)
    {
        public DbSet<Player> Players { get; set; }
    }
}
```

Repositories/IPlayerRepository

```
using WebApp4_By_Sushanka.Models;

namespace WebApp4_By_Sushanka.Repositories
{
    public interface IPlayerRepository
    {
        Task<List<Player>> GetAllPlayersAsync();
        Task<Player?> GetPlayerByIdAsync(int id);
        Task AddPlayerAsync(Player p);
        Task UpdatePlayerAsync(Player p);
        Task DeletePlayerAsync(int id);
    }
}
```

Repositories/PlayerRepository

```
using Microsoft.EntityFrameworkCore;
using WebApp4_By_Sushanka.Data;
using WebApp4_By_Sushanka.Models;

namespace WebApp4_By_Sushanka.Repositories
{
    public class PlayerRepository(PlayerDbContext context) : IPlayerRepository
    {
        private readonly PlayerDbContext _context = context;

        public async Task<List<Player>> GetAllPlayersAsync()
        {
            return await _context.Players.ToListAsync();
        }

        public async Task<Player?> GetPlayerByIdAsync(int id)
        {
            return await _context.Players.FindAsync(id);
        }

        public async Task AddPlayerAsync(Player p)
        {
            await _context.Players.AddAsync(p);
            await _context.SaveChangesAsync();
        }

        public async Task UpdatePlayerAsync(Player p)
        {
            _context.Players.Update(p);
            await _context.SaveChangesAsync();
        }

        public async Task DeletePlayerAsync(int id)
        {
            var player = await _context.Players.FindAsync(id) ?? null;
            if (player == null) return;
            _context.Players.Remove(player);
            await _context.SaveChangesAsync();
        }
    }
}
```

Program.cs

```
// Add DbContext with SQL Server provider
builder.Services.AddDbContext<PlayerDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("conStr")));
// Register the repository using dependency injection
builder.Services.AddScoped<IPlayerRepository, PlayerRepository>();
```

appsettings.json

```
"ConnectionStrings": {
  "conStr": "Server=localhost; Database=WebApp4_CodeFirst;
Trusted_Connection=True; TrustServerCertificate=True;"
}
```


Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using WebApp4_By_Sushanka.Models;
using WebApp4_By_Sushanka.Repositories;

namespace WebApp4_By_Sushanka.Controllers
{
    public class HomeController(IPlayerRepository playerRepository) : Controller
    {
        private readonly IPlayerRepository _player = playerRepository;

        public async Task<IActionResult> Index()
        {
            var players = await _player.GetAllPlayersAsync();
            return View(players);
        }

        public async Task<IActionResult> ViewPlayer(int id)
        {
            var player = await _player.GetPlayerByIdAsync(id);
            return View(player);
        }

        public IActionResult AddPlayer()
        {
            return View();
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> AddPlayer(Player p)
        {
            if (ModelState.IsValid)
            {
                await _player.AddPlayerAsync(p);
                return RedirectToAction("Index");
            }
            return Content("Failed to register player.");
        }

        public async Task<IActionResult> EditPlayer(int id)
        {
            var player = await _player.GetPlayerByIdAsync(id);
            return View(player);
        }

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> EditPlayer(Player p)
        {
            if (ModelState.IsValid)
            {
                await _player.UpdatePlayerAsync(p);
                return RedirectToAction("Index");
            }
            return Content("Failed to update player");
        }

        public async Task<IActionResult> DeletePlayer(int id)
        {
            await _player.DeletePlayerAsync(id);
            return RedirectToAction("Index");
        }
    }
}
```

Scaffolds Views

Views/Home/Index.cshtml

```
@model IEnumerable<WebApp4_By_Sushanka.Models.Player>

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<p>
    <a asp-action="AddPlayer">Create New</a>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Name)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Country)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Club)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Position)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Foot)
            </th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model) {
            <tr>
                <td>
                    @Html.DisplayFor(modelItem => item.Name)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Country)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Club)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Position)
                </td>
                <td>
                    @Html.DisplayFor(modelItem => item.Foot)
                </td>
                <td>
                    <a asp-action="EditPlayer" asp-route-id="@item.Id">Edit</a> |
                    <a asp-action="ViewPlayer" asp-route-id="@item.Id">Details</a> |
                    <a asp-action="DeletePlayer" asp-route-id="@item.Id">Delete</a>
                </td>
            </tr>
        }
    </tbody>
</table>
```

Views/Home/ViewPlayer.cshtml

```
@model WebApp4_By_Sushanka.Models.Player

@{
    ViewData["Title"] = "ViewPlayer";
}

<h1>ViewPlayer</h1>

<div>
    <h4>Player</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Name)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Name)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Country)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Country)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Club)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Club)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Position)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Position)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Foot)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Foot)
        </dd>
    </dl>
</div>
<div>
    <a asp-action="EditPlayer" asp-route-id="@Model?.Id">Edit</a> |
    <a asp-action="Index">Back to List</a>
</div>
```

Views/Home/AddPlayer.cshtml

```
@model WebApp4_By_Sushanka.Models.Player

@{
    ViewData["Title"] = "AddPlayer";
}

<h1>AddPlayer</h1>

<h4>Player</h4>
<hr />
```

```

<div class="row">
  <div class="col-md-4">
    <form asp-action="AddPlayer">
      <div asp-validation-summary="ModelOnly" class="text-danger"></div>
      <div class="form-group">
        <label asp-for="Name" class="control-label"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="Country" class="control-label"></label>
        <input asp-for="Country" class="form-control" />
        <span asp-validation-for="Country" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="Club" class="control-label"></label>
        <input asp-for="Club" class="form-control" />
        <span asp-validation-for="Club" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="Position" class="control-label"></label>
        <input asp-for="Position" class="form-control" />
        <span asp-validation-for="Position" class="text-danger"></span>
      </div>
      <div class="form-group">
        <label asp-for="Foot" class="control-label"></label>
        <input asp-for="Foot" class="form-control" />
        <span asp-validation-for="Foot" class="text-danger"></span>
      </div>
      <div class="form-group">
        <input type="submit" value="Create" class="btn btn-primary" />
      </div>
    </form>
  </div>
</div>

<div>
  <a asp-action="Index">Back to List</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}

```

Views/Home/EditPlayer.cshtml

```

@model WebApp4_By_Sushanka.Models.Player

@{
    ViewData["Title"] = "EditPlayer";
}

<h1>EditPlayer</h1>

<h4>Player</h4>
<hr />
<div class="row">
  <div class="col-md-4">
    <form asp-action="EditPlayer">
      <div asp-validation-summary="ModelOnly" class="text-danger"></div>
      <input type="hidden" asp-for="Id" />
      <div class="form-group">

```

```

        <label asp-for="Name" class="control-label"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Country" class="control-label"></label>
        <input asp-for="Country" class="form-control" />
        <span asp-validation-for="Country" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Club" class="control-label"></label>
        <input asp-for="Club" class="form-control" />
        <span asp-validation-for="Club" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Position" class="control-label"></label>
        <input asp-for="Position" class="form-control" />
        <span asp-validation-for="Position" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Foot" class="control-label"></label>
        <input asp-for="Foot" class="form-control" />
        <span asp-validation-for="Foot" class="text-danger"></span>
    </div>
    <div class="form-group">
        <input type="submit" value="Save" class="btn btn-primary" />
    </div>
</form>
</div>
</div>

<div>
    <a asp-action="Index">Back to List</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}

```

Views/Home/DeletePlayer.cshtml

```

@model WebApp4_By_Sushanka.Models.Player

@{
    ViewData["Title"] = "DeletePlayer";
}

<h1>DeletePlayer</h1>

<h3>Are you sure you want to delete this?</h3>
<div>
    <h4>Player</h4>
    <hr />
    <dl class="row">
        <dt class = "col-sm-2">
            @Html.DisplayNameFor(model => model.Name)
        </dt>
        <dd class = "col-sm-10">
            @Html.DisplayFor(model => model.Name)
        </dd>
        <dt class = "col-sm-2">
            @Html.DisplayNameFor(model => model.Country)

```

```

</dt>
<dd class = "col-sm-10">
    @Html.DisplayFor(model => model.Country)
</dd>
<dt class = "col-sm-2">
    @Html.DisplayNameFor(model => model.Club)
</dt>
<dd class = "col-sm-10">
    @Html.DisplayFor(model => model.Club)
</dd>
<dt class = "col-sm-2">
    @Html.DisplayNameFor(model => model.Position)
</dt>
<dd class = "col-sm-10">
    @Html.DisplayFor(model => model.Position)
</dd>
<dt class = "col-sm-2">
    @Html.DisplayNameFor(model => model.Foot)
</dt>
<dd class = "col-sm-10">
    @Html.DisplayFor(model => model.Foot)
</dd>
</dl>

<form asp-action="DeletePlayer">
    <input type="hidden" asp-for="Id" />
    <input type="submit" value="Delete" class="btn btn-danger" /> |
    <a asp-action="Index">Back to List</a>
</form>
</div>

```

AddPlayer

Player

Name

Ngolo Kante

Country

France

Club

Chelsea

Position

DM

Foot

Right

Create

[Back to List](#)

Index

[Create New](#)

Name	Country	Club	Position	Foot	
Messi	Argentina	Barca	LW	Left	Edit Details Delete
Ngolo Kante	France	Chelsea	DM	Right	Edit Details Delete

EditPlayer

Player

Name

Xavi

Country

Spain

Club

None

Position

CM

Foot

Right

Save

[Back to List](#)

Index

[Create New](#)

Name	Country	Club	Position	Foot	
Messi	Argentina	Barca	RW	Left	Edit Details Delete
Ngolo Kante	France	Chelsea	DM	Right	Edit Details Delete
Xavi	Spain	None	CM	Right	Edit Details Delete

23. EF Core DB First

Sql query

```
CREATE DATABASE WebApp5_DbFirst;
GO

USE WebApp5_DbFirst;

CREATE TABLE Students (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(100) NOT NULL,
    Age INT NOT NULL
);

INSERT INTO Students (Name, Age) VALUES
('Pralad', 20),
('Jaya', 22),
('Ghaman Singh', 19),
('Naradev', 23),
('Devendra', 21);
```

On Packet Manager Console

```
Scaffold-DbContext "Server=localhost; Database= WebApp5_DbFirst;
Trusted_Connection=True; TrustServerCertificate=True;"
Microsoft.EntityFrameworkCore.SqlServer -o Models;

Or.....

PM> scaffold-dbcontext "Name=conStr" Microsoft.EntityFrameworkCore.SqlServer -o
Models
```

Created Models

Models/Student.cs

```
using System;
using System.Collections.Generic;

namespace WebApp5_By_Sushanka.Models;

public partial class Student
{
    public int Id { get; set; }

    public string Name { get; set; } = null!;

    public int Age { get; set; }
}
```

Models/WebApp5DbFirstContext.cs

```
using Microsoft.EntityFrameworkCore;

namespace WebApp5_By_Sushanka.Models;

public partial class WebApp5DbFirstContext : DbContext
{
    public WebApp5DbFirstContext()
    {
    }
```



```

    }

    public WebApp5DbFirstContext(DbContextOptions<WebApp5DbFirstContext> options)
        : base(options)
    {
    }

    public virtual DbSet<Student> Students { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        => optionsBuilder.UseSqlServer("Name=conStr");

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Student>(entity =>
        {
            entity.HasKey(e => e.Id).HasName("PK__Students__3214EC07A394FE04");

            entity.Property(e => e.Name).HasMaxLength(100);
        });

        OnModelCreatingPartial(modelBuilder);
    }

    partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}

```

Create Controller with view using Scaffolding

Controllers/StudentsController.cs

```

using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using WebApp5_By_Sushanka.Models;

namespace WebApp5_By_Sushanka.Controllers
{
    public class StudentsController : Controller
    {
        private readonly WebApp5DbFirstContext _context;

        public StudentsController(WebApp5DbFirstContext context)
        {
            _context = context;
        }

        // GET: Students
        public async Task<IActionResult> Index()
        {
            return View(await _context.Students.ToListAsync());
        }

        // GET: Students/Details/5
        public async Task<IActionResult> Details(int? id)
        {
            if (id == null)
            {
                return NotFound();
            }

            var student = await _context.Students
                .FirstOrDefaultAsync(m => m.Id == id);

```

```

        if (student == null)
        {
            return NotFound();
        }

        return View(student);
    }

    // GET: Students/Create
    public IActionResult Create()
    {
        return View();
    }

    // POST: Students/Create
    // To protect from overposting attacks, enable the specific properties you
    want to bind to.
    // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Create([Bind("Id,Name,Age")] Student
student)
    {
        if (ModelState.IsValid)
        {
            _context.Add(student);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        return View(student);
    }

    // GET: Students/Edit/5
    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)
        {
            return NotFound();
        }

        var student = await _context.Students.FindAsync(id);
        if (student == null)
        {
            return NotFound();
        }
        return View(student);
    }

    // POST: Students/Edit/5
    // To protect from overposting attacks, enable the specific properties you
    want to bind to.
    // For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(int id, [Bind("Id,Name,Age")]
Student student)
    {
        if (id != student.Id)
        {
            return NotFound();
        }

        if (ModelState.IsValid)
        {
            try

```

```

        {
            _context.Update(student);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!StudentExists(student.Id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }
        return RedirectToAction(nameof(Index));
    }
    return View(student);
}

// GET: Students/Delete/5
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var student = await _context.Students
        .FirstOrDefaultAsync(m => m.Id == id);
    if (student == null)
    {
        return NotFound();
    }

    return View(student);
}

// POST: Students/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var student = await _context.Students.FindAsync(id);
    if (student != null)
    {
        _context.Students.Remove(student);
    }

    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}

private bool StudentExists(int id)
{
    return _context.Students.Any(e => e.Id == id);
}
}

```

Index

[Create New](#)

Name	Age	
Pralad	20	Edit Details Delete
Jaya	22	Edit Details Delete
Ghaman Singh	19	Edit Details Delete
Naradev	23	Edit Details Delete
Devendra	21	Edit Details Delete

Create Student

Name

Niroj

Age

22

Create

[Back to List](#)

Index

[Create New](#)

Name	Age	
Pralad	20	Edit Details Delete
Jaya	22	Edit Details Delete
Ghaman Singh	19	Edit Details Delete
Naradev	23	Edit Details Delete
Devendra	21	Edit Details Delete
Niroj	22	Edit Details Delete

Edit Student

Name

Ghaman Singh

Age

29

Save

[Back to List](#)

Delete

Are you sure you want to delete this?

Student

Name

Devendra

Age

21

Delete

| [Back to List](#)

Index

[Create New](#)

Name	Age	
Pralad	20	Edit Details Delete
Jaya	22	Edit Details Delete
Ghaman Singh	29	Edit Details Delete
Naradev	23	Edit Details Delete
Niroj	22	Edit Details Delete

24.WebApi

Initially, Asp.Net Web Api template only has Controllers.

Models/Student.cs

```
namespace WebApi_By_Sushanka.Models
{
    public class Student
    {
        public int Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public int Age { get; set; }
        public string Email { get; set; } = string.Empty;
    }
}
```

Data/ApiDbContext

```
using Microsoft.EntityFrameworkCore;
using System.Collections.Generic;
using WebApi_By_Sushanka.Models;

namespace WebApi_By_Sushanka.Data
{
    public class ApiDbContext(DbContextOptions<ApiDbContext> options)
    : DbContext(options)
    {
        public DbSet<Student> Students { get; set; }
    }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using WebApi_By_Sushanka.Data;

namespace WebApi_By_Sushanka
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.

            builder.Services.AddControllers();
            // Learn more about configuring OpenAPI at
https://aka.ms/aspnet/openapi
            builder.Services.AddOpenApi();

            builder.Services.AddEndpointsApiExplorer();
```

```

        builder.Services.AddSwaggerGen();

        // add EF Core with SQL Server
        builder.Services.AddDbContext<ApiDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("conStr")));

        var app = builder.Build();

        // Configure the HTTP request pipeline.
        if (app.Environment.IsDevelopment())
        {
            app.MapOpenApi();
            app.UseSwagger();
            app.UseSwaggerUI();
        }

        app.UseHttpsRedirection();
        app.UseAuthorization();
        app.MapControllers();

        app.Run();
    }
}

```

appsettings.json

```

"ConnectionStrings": {
  "conStr": "Server=localhost; Database=Api_Db; Trusted_Connection=True;
TrustServerCertificate=True;"
}

```

Controllers/StudentApiController.cs

```

using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using WebApi_By_Sushanka.Data;
using WebApi_By_Sushanka.Models;

namespace WebApi_By_Sushanka.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class StudentApiController(ApiDbContext context) : ControllerBase
    {
        private readonly ApiDbContext _context= context;

        // GET: api/students
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Student>>> GetStudents()
        {
            return await _context.Students.ToListAsync();
        }

        // GET: api/students/3
        [HttpGet("{id}")]
        public async Task<ActionResult<Student>> GetStudent(int id)
        {
            var student = await _context.Students.FindAsync(id);
            if (student == null)

```



```

        return NotFound();
    }
    return student;
}

// POST: api/students/3
[HttpPost]
public async Task<ActionResult<Student>> PostStudent(Student student)
{
    student.Id = 0; // now EF knows it's a new entity
    _context.Students.Add(student);
    await _context.SaveChangesAsync();
    return CreatedAtAction(nameof(GetStudent), new { id = student.Id },
student);
}

// PUT: api/students/3
[HttpPut("{id}")]
public async Task<IActionResult> PutStudent(int id, Student student)
{
    if (id != student.Id) return BadRequest();
    _context.Entry(student).State = EntityState.Modified;
    await _context.SaveChangesAsync();
    return NoContent();
}

// DELETE: api/students/3
[HttpDelete("{id}")]
public async Task<IActionResult> DeleteStudent(int id)
{
    var student = await _context.Students.FindAsync(id);
    if (student == null) return NotFound();
    _context.Students.Remove(student);
    await _context.SaveChangesAsync();
    return NoContent();
}
}
}

```

WebApi_By_Sushanka

1.0

OAS 3.0

<https://localhost:7018/swagger/v1/swagger.json>

StudentApi

GET	/api/StudentApi	▼
POST	/api/StudentApi	▼
GET	/api/StudentApi/{id}	▼
PUT	/api/StudentApi/{id}	▼
DELETE	/api/StudentApi/{id}	▼

POST

/api/StudentApi



Parameters

Cancel

Reset

No parameters

Request body

application/json



Edit Value | Schema

```
{
  "id": 0,
  "name": "Sushanka",
  "age": 23,
  "email": "sushanka@gmail.com"
}
```

Request URL

https://localhost:7018/api/StudentApi

Server response

Code

Details

201

Undocumented

Response body

```
{
  "id": 1,
  "name": "Sushanka",
  "age": 23,
  "email": "sushanka@gmail.com"
}
```



Download

Request URL

https://localhost:7018/api/StudentApi

Server response

Code

Details

200

Response body

```
[
  {
    "id": 1,
    "name": "Sushanka",
    "age": 23,
    "email": "sushanka@gmail.com"
  },
  {
    "id": 2,
    "name": "Kumar",
    "age": 23,
    "email": "kumar@gmail.com"
  }
]
```



Download

25. State Management

Models/User.cs

```
using System.ComponentModel.DataAnnotations;

namespace WebApp6_By_Sushanka.Models
{
    public class User
    {
        [Required]
        public int Id { get; set; }
        [Required]
        public string Name { get; set; }
        [Required]
        public string Role { get; set; }
    }
}
```

Controllers/HomeController.cs

```
using System.Diagnostics;
using Microsoft.AspNetCore.Localization;
using Microsoft.AspNetCore.Mvc;
using WebApp6_By_Sushanka.Models;

namespace WebApp6_By_Sushanka.Controllers
{
    public class HomeController : Controller
    {
        // Dummy ceredentials
        private readonly string USERNAME = "admin";
        private readonly string PASSWORD = "123";

        public IActionResult Login()
        {
            // check if user already logged in; redirect to home/index
            if (HttpContext.Session.GetString("Username") != null)
            {
                return RedirectToAction("Index");
            }

            if (Request.Cookies.ContainsKey("SavedUsername"))
            {
                ViewBag.SavedUsername = Request.Cookies["SavedUsername"];
            }

            return View();
        }

        [HttpPost]
        public IActionResult Login(string Username, string Password, bool rememberMe)
        // razor and model binding are case insensitive
        {
            Console.WriteLine("Login POST called");
            foreach (var item in Request.Form.Keys)
            {
                Console.WriteLine($"{item} : {Request.Form[item]}");
            }
            Console.WriteLine($"Username: {Username}, Password: {Password},
savedUsername: {rememberMe}");
            if (Username == USERNAME && Password == PASSWORD)
            {
                // setting session
            }
        }
    }
}
```

```

        HttpContext.Session.SetString("Username", Username);
        //HttpContext.Session.SetString("LoginTime",
DateTime.Now.ToString("mm:ss"));
        HttpContext.Session.SetString("LoginTime",
DateTime.Now.ToString("yyyy-MM-ddTHH:mm:ss"));

        if (rememberMe)
        {
            Response.Cookies.Append("SavedUsername", Username, new
CookieOptions
            {
                Expires = DateTime.Now.AddDays(7) // Cookie will expire in 7
days
            });
        }
        else // delete cookie if user uncheck the checkbox
        {
            Response.Cookies.Delete("SavedUsername");
        }
        return RedirectToAction("Index");
    }
    else
    {
        ViewBag.Error = "Invalid Username or Password!!!";
        return View();
    }
}

public IActionResult Logout()
{
    return View();
}

[HttpPost]
public IActionResult LogOut()
{
    // clear session
    HttpContext.Session.Clear();
    return RedirectToAction("Index");
}

public IActionResult Index()
{
    if (HttpContext.Session.GetString("Username") == null)
    {
        return RedirectToAction("Login");
    }
    return View();
}

[HttpPost]
public IActionResult Index(IFormCollection form)
{
    var username = form["username"].ToString();
    CookieOptions options = new CookieOptions();
    options.Expires = DateTime.Now.AddSeconds(10); // Cookie will expire in
10 seconds
    Response.Cookies.Append("username", username, options);
    //Response.Cookies.Append("Email", "User@gmail.com", new CookieOptions {
Expires = DateTime.Now.AddSeconds(5) });
    return View();
}

public IActionResult GetCookies()
{

```

```

        // Retrieving a cookie
        // ViewBag.CookieValue = Request.Cookies["username"] ?? "Cookie not found";
// shortcut of HttpContext.Request.Cookies
TempData["CookieValue"] = Request.Cookies["username"] ?? "Cookie not
found";
        // shortcut of HttpContext.Request.Cookies
//return View("Index");
return RedirectToAction("Index");
    }

    public IActionResult DeleteCookies()
    {
        // Deleting a cookie
        Response.Cookies.Delete("username");
        TempData["CookieValue"] = "Cookie deleted";
        return RedirectToAction("Index");
    }

    public IActionResult GetUser(int id, string name, string role)
    {
        User newUser = new User()
        {
            Id = id,
            Name = name,
            Role = role,
        };
        if (Request.Cookies.ContainsKey("Username")) // only logged in user can
access this action
        {
            return View(newUser);
        }
        return RedirectToAction("Login");
    }
}

```

Views/Shared/ Layout.html

```

<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
    <ul class="navbar-nav ms-auto">
        @{
            var username = Context.Session.GetString("Username");
            if (username != null){
                <li class="nav-item">
                    <a class="nav-link text-success">
                        Welcome, @username<br />
                        Your session expires in <span id="remaining-time" class="btn-
primary"></span>
                    </a>
                </li>
                <li class="nav-item">
                    <a class="nav-link text-dark" asp-area="" asp-controller="Home"
asp-action="Logout">Logout</a>
                </li>
            }
            else
            {
                <li class="nav-item">
                    <a class="nav-link text-dark" asp-area="" asp-controller="Home"
asp-action="Login">Login</a>
                </li>
            }
        }
    </ul>

```



```

        <a asp-action="GetCookies" class="btn btn-success me-2">Read
Cookies</a>
        <a asp-action="DeleteCookies" class="btn btn-danger">Delete
Cookies</a>
    </div>

    @if (TempData["CookieValue"] != null)
    {
        <div class="alert alert-info mt-3 text-center">
            Cookie Value: @TempData["CookieValue"]
        </div>
    }
</div>

<div class="mt-4">
    <partial name="_CreateUser.cshtml" />
</div>
</div>
</div>
</div>

@section Scripts {
    @{
        await Html.RenderPartialAsync("_ValidationScriptsPartial");
    }
}

```

Views/Home/ CreateUser.cshtml

@model WebApp6_By_Sushanka.Models.User

```

<h4>User</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="GetUser" method="get">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="form-group">
                <label asp-for="Id" class="control-label"></label>
                <input asp-for="Id" class="form-control" />
                <span asp-validation-for="Id" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Name" class="control-label"></label>
                <input asp-for="Name" class="form-control" />
                <span asp-validation-for="Name" class="text-danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Role" class="control-label"></label>
                <input asp-for="Role" class="form-control" />
                <span asp-validation-for="Role" class="text-danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Create" class="btn btn-primary" />
            </div>
        </form>
    </div>
</div>

<div>
    <a asp-action="Index">Back to List</a>
</div>

```


Views/Home/GetUser.cshtml

```
@model WebApp6_By_Sushanka.Models.User

@{
    ViewData["Title"] = "User Details";
}

<h1>User Details</h1>

<div>
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Id)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Id)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Name)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Name)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Role)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Role)
        </dd>
    </dl>
</div>
<div>
    <a asp-action="Index">Back to List</a>
</div>
```

Views/Home/Login.cshtml

```
@{
    ViewBag.Title = "Login";
}

<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-5">
            <div class="card shadow-sm">
                <div class="card-body">
                    <h3 class="card-title text-center mb-4">Login</h3>

                    <form action="Login" method="post">
                        <div class="mb-3">
                            <label for="username" class="form-label">Username:</label>
                            <input name="username" value="@ViewBag.SavedUsername"
                                id="username" class="form-control" required />
                        </div>
                        <div class="mb-3">
```

```

        <label for="password" class="form-
label">Password:</label>
        <input name="password" id="password" type="password"
class="form-control" required />
    </div>
    <div class="form-check mb-3">
        <input name="rememberMe" type="checkbox"
@(<ViewBag.SavedUsername != null ? "checked" : ""> value="true" class="form-check-
input" id="rememberMe" /> @*if value is not set it will return "on" & it will not
be converted to bool*@
        <label class="form-check-label"
for="rememberMe">Remember Username</label>
    </div>
    <div class="text-center">
        <input type="submit" value="Login" class="btn btn-
primary w-100" />
    </div>
</form>

@if (ViewBag.Error != null)
{
    <div class="alert alert-danger mt-3 text-center">
        @ViewBag.Error
    </div>
}
</div>
</div>
</div>
</div>
</div>

```

Views/Home/Logout.cshtml

```

@{
    ViewData["Title"] = "Logout Confirmation";
}

<div class="container mt-5">
    <div class="row justify-content-center">
        <div class="col-md-5">
            <div class="card shadow-sm">
                <div class="card-body text-center">
                    <h3 class="card-title mb-4">Are you sure you want to
logout?</h3>
                    <p class="mb-4">You will be logged out and your session will
end.</p>
                    <div class="d-flex justify-content-center gap-3">
                        <form asp-action="Logout" method="post">
                            <button type="submit" class="btn btn-danger">Yes,
Logout</button>
                        </form>
                        <a asp-action="Index" class="btn btn-secondary">No, Stay
Logged In</a>
                    </div>
                </div>
            </div>
        </div>
    </div>
</div>
</div>

```

Login

Username:

Password:

☒ Remember Username

Login

Welcome

Enter your name to store in cookies:

Set Cookie

Read Cookies Delete Cookies

User

Id

Name

Role

Create

[Back to List](#)

Welcome

Enter your name to store in cookies:

Set Cookie

Read Cookies

Delete Cookies

Cookie Value: Sushanka

Welcome

Enter your name to store in cookies:

Set Cookie

Read Cookies

Delete Cookies

Cookie Value: Cookie deleted

User

Id

10

Name

Sushanka

Role

Admin

Create

[Back to List](#)

Are you sure you want to logout?

You will be logged out and your session will end.

Yes, Logout

No, Stay Logged In

Login

Username:

admin

Password:

☒ Remember Username

Login

26. Client side development in ASP.Net core

Angular+React App By Sushanka

[theory.docx](#)

Angular vs React Setup for .NET

Angular Standalone	react standalone
ng new Angular_Standalone cd Angular_Standalone ng serve	npx create-react-app react_strandalone cd react_standalone npm start
ng generate component components/navbar ng generate component components/home ng generate component components/calculator ng generate component components/footer	<u>npm install bootstrap</u> On index.js <u>import 'bootstrap/dist/css/bootstrap.min.css';</u>
npm install @angular/router	

Add bootstrap to index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>AngularStandalone</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" />
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

add assets to angular.json

```
"build": {
  "builder": "@angular/build:application",
  "options": {
    "browser": "src/main.ts",
    "polyfills": [
      "zone.js"
    ],
    "tsConfig": "tsconfig.app.json",
    "assets": [
      "src/assets",
      "src/favicon.ico",
      {
        "glob": "**/*",
        "input": "public"
      }
    ]
  },
}
```

Angular Standalone

app/components/home, navbar, footer, calculator

navbar.html

```
<nav class="navbar navbar-expand-lg navbar-light bg-light">
  <a class="navbar-brand">MyApp</a>
  <div class="collapse navbar-collapse">
    <ul class="navbar-nav mr-auto">
      <li class="nav-item">
        <a class="nav-link" routerLink="/">Home</a>
      </li>
      <li class="nav-item">
        <a class="nav-link" routerLink="/calculator">Calculator</a>
      </li>
      <li class="nav-item">
        <!-- Normal hyperlink to .NET controller -->
        <a class="nav-link" href="/Home/Index">Go to .NET Home</a>
      </li>
      <li class="nav-item">
        <!-- Normal hyperlink to .NET controller -->
        <a class="nav-link" href="/react">Go to React Home</a>
      </li>
    </ul>
  </div>
</nav>
```

navbar.ts

```
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-navbar',
  standalone: true,
  imports: [RouterModule],
  templateUrl: './navbar.html',
  styleUrls: ['./navbar.css'],
})
export class Navbar {}
```

home.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  templateUrl: './home.html',
  standalone: true,
})
export class Home {}
```

home.html

```
<div class="text-center mt-5">
  <h2>Welcome to Home Page</h2>
  
</div>
```

calculator.html

```
<div class="container mt-5">
  <h2>Calculator</h2>
  <div class="form-group">
    <input type="number" class="form-control" placeholder="Enter first number"
    [(ngModel)]="num1" />
  </div>
  <div class="form-group mt-2">
    <input type="number"
      class="form-control"
      placeholder="Enter second number"
      [(ngModel)]="num2" />
  </div>
  <div class="form-group mt-2">
    <select class="form-control" [(ngModel)]="operation">
```

<pre> <option value="add">Add</option> <option value="subtract">Subtract</option> <option value="multiply">Multiply</option> </select> </div> <button class="btn btn-primary mt-2" (click)="compute()">Compute</button> <h3 class="mt-3">Result: {{ result }}</h3> </div> </pre>	
footer.html	
<pre> <footer class="text-center mt-5">@SushankaKhadka, {{ currentYear }}</footer> </pre>	

calculator.ts	footer.ts
<pre> import { Component } from '@angular/core'; import { FormsModule } from '@angular/forms'; @Component({ selector: 'app-calculator', templateUrl: './calculator.html', standalone: true, imports: [FormsModule], // needed for [(ngModel)] }) export class Calculator { num1 = 0; num2 = 0; operation = 'add'; result = 0; compute() { switch (this.operation) { case 'add': this.result = this.num1 + this.num2; break; case 'subtract': this.result = this.num1 - this.num2; break; case 'multiply': this.result = this.num1 * this.num2; break; } } } </pre>	<pre> import { Component } from '@angular/core'; @Component({ selector: 'app-footer', templateUrl: './footer.html', standalone: true, }) export class Footer { currentYear = new Date().getFullYear(); } </pre>
app.route.ts	
<pre> import { Routes } from '@angular/router'; export const routes: Routes = []; </pre>	

app.ts	app.html
<pre> import { Component, signal } from '@angular/core'; import { RouterOutlet } from '@angular/router'; import { Navbar } from './components/navbar/navbar'; import { Footer } from './components/footer/footer'; @Component({ selector: 'app-root', templateUrl: './app.html', styleUrls: ['./app.css'], standalone: true, imports: [RouterModule, Navbar, Footer], }) export class App { </pre>	<pre> <app-navbar></app-navbar> <div class="container"> <router-outlet></router-outlet> </div> <app-footer></app-footer> </pre>

<pre> protected readonly title = signal('Angular_Standalone'); } </pre>	
main.ts	
<pre> import { bootstrapApplication } from '@angular/platform-browser'; import { App } from './app/app'; import { Home } from './app/components/home/home'; import { Calculator } from './app/components/calculator/calculator'; import { provideRouter } from '@angular/router'; bootstrapApplication(App, { providers: [provideRouter([{ path: '', component: Home }, { path: 'calculator', component: Calculator }, { path: '**', redirectTo: '' }, // fallback route]),], }); </pre>	

React builup

Components/Calculator.js, Footer.js, Home.js, Navbar.js	
Calculator.js	
<pre> // src/components/Calculator.js import { useState } from 'react'; export default function Calculator() { const [num1, setNum1] = useState(''); const [num2, setNum2] = useState(''); const [operation, setOperation] = useState('add'); const [result, setResult] = useState(null); const handleCompute = (e) => { e.preventDefault(); const a = parseFloat(num1); const b = parseFloat(num2); let res = 0; if (operation === 'add') res = a + b; if (operation === 'subtract') res = a - b; if (operation === 'multiply') res = a * b; setResult(res); }; return (<div className="container mt-5" style={{ maxWidth: '400px' }}> <h3>Calculator</h3> <form onSubmit={handleCompute} className="border p-4 shadow rounded bg-light"> <div className="mb-3"> <label>Number 1</label> <input type="number" className="form-control" value={num1} onChange={e => setNum1(e.target.value)} required /> </div> <div className="mb-3"> <label>Number 2</label> <input type="number" className="form-control" value={num2} onChange={e => setNum2(e.target.value)} required /> </div> </pre>	

```

        <div className="mb-3">
          <label>Operation</label>
          <select className="form-select" value={operation} onChange={e =>
setOperation(e.target.value)}>
            <option value="add">Add</option>
            <option value="subtract">Subtract</option>
            <option value="multiply">Multiply</option>
          </select>
        </div>
        <button type="submit" className="btn btn-primary w-100">Compute</button>
      </form>
      {result !== null && <div className="mt-3 alert alert-success">Result: {result}</div>}
    </div>
  );
}

```

Footer.js

```

// src/components/Footer.js
export default function Footer() {
  const year = new Date().getFullYear();
  return (
    <footer className="bg-dark text-white text-center p-3 mt-auto">
      &copy; Sushanka Khadka {year}
    </footer>
  );
}

```

Home.js

```

// src/components/Home.js
import myPhoto from '../assets/tiger.jpg'; // put your photo in src folder

export default function Home() {
  return (
    <div className="text-center mt-5">
      <h2>Welcome to Home Page</h2>
      <img src={myPhoto} alt="myPhoto" className="img-thumbnail" style={{ maxWidth: '300px' }} />
    </div>
  );
}

```

Navbar.js

```

// src/components/Navbar.js
import { Link } from 'react-router-dom';

export default function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark shadow-sm">
      <div className="container">
        <Link className="navbar-brand" to="/">MyApp</Link>
        <div className="collapse navbar-collapse">
          <ul className="navbar-nav ms-auto">
            <li className="nav-item">
              <Link className="nav-link text-light" to="/">Home</Link>
            </li>
            <li className="nav-item">
              <Link className="nav-link text-light" to="/calculator">Calculator</Link>
            </li>
            <li className="nav-item">
              <a className="nav-link text-light" href="/Home/Index">Go to .NET Home</a>
            </li>
            <li className="nav-item">
              <a className="nav-link text-light" href="/angular">Go to Angular Home</a>
            </li>

```

```

        </ul>
      </div>
    </div>
  </nav>
);
}

```

app.js

```

import './App.css';
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import Home from './components/Home';
import Calculator from './components/Calculator';

function App() {
  return (
    <div className="app-container">
      <Navbar />
      <div className="content-wrap">
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/calculator" element={<Calculator />} />
        </Routes>
      </div>
      <Footer />
    </div>
  );
}

export default App;

```

index.js

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';
import { BrowserRouter } from 'react-router-dom';
import 'bootstrap/dist/css/bootstrap.min.css';

const isProduction = process.env.NODE_ENV === "production";

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <BrowserRouter basename={isProduction ? "/react" : "/"}>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);

// If you want to start measuring performance in your app, pass a function
// to log results (for example: reportWebVitals(console.log))
// or send to an analytics endpoint. Learn more: https://bit.ly/CRA-vitals
reportWebVitals();

```

Asp.Net WebApp

Angular+React App By Sushanka.proj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>Angular_React_App_By_Sushanka</RootNamespace>
  </PropertyGroup>

  <Target Name="PublishAngular" AfterTargets="Build">
    <!-- Install dependencies -->
    <Exec Command="npm install" WorkingDirectory=" ../Angular_Standalone " />

    <!-- Build Angular for production -->
    <Exec Command="npm run build -- --configuration=production --base-
href=/angular/" WorkingDirectory=" ../Angular_Standalone " />

    <ItemGroup>
      <AngularDistFiles
Include=" ../Angular_Standalone/dist/Angular_Standalone/browser/**" />
    </ItemGroup>

    <!-- Copy to wwwroot -->
    <Copy SourceFiles="@ (AngularDistFiles)"
DestinationFolder="wwwroot/angular/% (RecursiveDir)" SkipUnchangedFiles="true"/>
  </Target>

  <Target Name="PublishReact" AfterTargets="Build">
    <!-- Install dependencies -->
    <Exec Command="npm install" WorkingDirectory=" ../react_standalone " />
    <!-- Build React for production -->
    <!-- <Exec Command="npm run build" WorkingDirectory=" ../react_standalone"
/> -->
    <Exec Command="npx cross-env PUBLIC_URL=/react npm run build"
WorkingDirectory=" ../react_standalone " />

    <ItemGroup>
      <ReactDistFiles Include=" ../react_standalone/build/**" />
    </ItemGroup>

    <Copy SourceFiles="@ (ReactDistFiles)"
DestinationFolder="wwwroot/react/% (RecursiveDir)" />
  </Target>
</Project>
```

HomeController

```
using System.Diagnostics;
using Angular_React_App_By_Sushanka.Models;
using Microsoft.AspNetCore.Mvc;

namespace Angular_React_App_By_Sushanka.Controllers
{
    public class HomeController : Controller
    {

```

```

    public IActionResult Index()
    {
        return View();
    }

    public IActionResult Privacy()
    {
        return View();
    }

    public IActionResult Signup()
    {
        return View();
    }
}

```

Views/Home/Signup.cshtml

```

@{
    ViewData["Title"] = "Sign Up";
}

<div class="container mt-5">
    <h3 class="text-center">Name: Sushanka Khadka | Roll no: 84/079 </h3>
    <h4 class="text-center mb-4">Sign Up Form</h4>

    <form id="signupForm" method="post" class="border p-4 shadow rounded bg-light">
        <div class="mb-3">
            <label for="name" class="form-label">Full Name</label>
            <input type="text" id="name" name="name" class="form-control" />
        </div>

        <div class="mb-3">
            <label for="email" class="form-label">Email Address</label>
            <input type="email" id="email" name="email" class="form-control" />
        </div>

        <div class="mb-3">
            <label for="password" class="form-label">Password</label>
            <input type="password" id="password" name="password" class="form-control" />
        </div>

        <div class="mb-3">
            <label for="confirmPassword" class="form-label">Confirm Password</label>
            <input type="password" id="confirmPassword" name="confirmPassword"
class="form-control" />
        </div>

        <div class="mb-3">
            <label for="phone" class="form-label">Phone Number</label>
            <input type="text" id="phone" name="phone" class="form-control" />
        </div>

        <button type="submit" class="btn btn-primary w-100">Sign Up</button>
    </form>
</div>

@section Scripts{
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

```

```

<script src="https://cdn.jsdelivr.net/npm/jquery-validation@1.19.5/dist/jquery.validate.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/jquery-validation-unobtrusive@3.2.12/dist/jquery.validate.unobtrusive.min.js"></script>

<script>
$(document).ready(function (){
    $("#signupForm").validate({
        rules: {
            name : {
                required: true,
                minlength: 3
            },
            email: {
                required: true,
                email: true
            },
            password: {
                required: true,
                minlength: 6
            },
            confirmPassword: {
                required: true,
                equalTo: "#password"
            },
            phone: {
                required: true,
                digits: true,
                minlength: 10,
                maxlength: 10
            }
        },
        messages: {
            name: {
                required: "Please enter your name.",
                minlength: "Name must be at lease 3 characters long."
            },
            email: {
                required: "Please enter an email",
                email: "Enter a valid email address"
            },
            password: {
                required: "Please provide a password.",
                minlength: "Password must be at lease 6 characters long"
            },
            confirmPassword: {
                required: "Please confirm password",
                equalTo: "Passwords do not match"
            },
            phone: {
                required: "Pleae enter your phone number",
                digits: "Only numbers allowed",
                minlength: "Phone must be 10 digits",
                maxlength: "Phone must be 10 digits"
            }
        },
        errorClass: "text-danger"
    });
});
</script>
}

```

program.cs

```
namespace Angular_React_App_By_Sushanka
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllersWithViews();

            var app = builder.Build();

            // Configure the HTTP request pipeline.
            if (!app.Environment.IsDevelopment())
            {
                app.UseExceptionHandler("/Home/Error");
                // The default HSTS value is 30 days. You may want to change this for
                production scenarios, see https://aka.ms/aspnetcore-hsts.
                app.UseHsts();
            }
            app.UseHttpsRedirection();

            // Serve Angular build files
            //app.UseDefaultFiles(); // looks for index.html in wwwroot
            app.UseStaticFiles(); // serves js/css/assets from wwwroot

            app.UseRouting();
            app.UseAuthorization();

            // Map API endpoints
            app.MapControllers(); // /api/* routes handled here

            // Map MVC routes
            app.MapControllerRoute(
                name: "default",
                pattern: "{controller=Home}/{action=Index}/{id?}");

            app.MapFallbackToFile("/angular/{*path:nonfile}", "angular/index.html");
            app.MapFallbackToFile("/react/{*path:nonfile}", "react/index.html");

            app.Run();
        }
    }
}
```

layout.cshtml

```
<div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
    <ul class="navbar-nav flex-grow-1">
        <li class="nav-item">
            <!-- Link to Angular Home -->
            <a class="nav-link text-dark" href="/angular">Go to Angular Home</a>
        </li>
        <li class="nav-item">
            <a class="nav-link text-dark" href="/react">Go to React Home</a>
        </li>
        <li class="nav-item">
            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-
            action="Signup">Signup</a>
        </li>
    </ul>
</div>
```

localhost:7171

Angular_React_App_By_Sushanka

Home

Privacy

Go to Angular Home

Go to React Home

Signup

Welcome

Learn about [building Web apps with ASP.NET Core](#).

© 2025 - Angular_React_App_By_Sushanka - [Privacy](#)

localhost:7171/angular/

MyApp

Home

Calculator

Go to .NET Home

Go to React Home

Welcome to Home Page



©SushankaKhadka, 2025

localhost:7171/react/calculator

MyApp

Home

Calculator

Go to .NET Home

Go to Angular Home

Calculator

Number 1

35

Number 2

-57

Operation

Multiply

▼

Compute

Result: -1995

© Sushanka Khadka 2025

Name: Sushanka Khadka | Roll no: 84/079

Sign Up Form

Full Name

Sushanka

Email Address

user@gmail.com

Password

.....

Confirm Password

.....

Passwords do not match

Phone Number

9800000000

Sign Up

27. Authentication & Authorization

HomeController

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using NuGet.Protocol.Plugins;
using System.Security.Claims;

namespace WebApp7_By_Sushanka.Controllers
{
    // [Authorize(Roles = "Admin")] // only admin can manage users
    public class HomeController(UserManager<IdentityUser> userManager,
    RoleManager<IdentityRole> roleManager) : Controller
    {
        private readonly UserManager<IdentityUser> _userManager =
userManager;
        private readonly RoleManager<IdentityRole> _roleManager =
roleManager;

        // List all users with roles/claims/policies
        [Authorize(Roles = "Admin")]
        public async Task<IActionResult> ManageUsers()
        {
            var users = await _userManager.Users.ToListAsync();
            var model = new List<UserViewModel>();

            foreach (var user in users)
            {
                var roles = await _userManager.GetRolesAsync(user);
                var claims = await _userManager.GetClaimsAsync(user);

                // check policies manually
                bool isAdminPolicy = roles.Contains("Admin");
                bool isHRPolicy = claims.Any(c => c.Type ==
"Department" && c.Value == "HR");
                bool isOver18Policy = claims.Any(c => c.Type == "Age"
&& int.TryParse(c.Value, out var age) && age >= 18);

                model.Add(new UserViewModel
                {
                    UserId = user.Id,
                    Email = user.Email,
                    Roles = roles,
                    Claims = [..claims.Select(c => $"{c.Type} :
{c.Value}")],
                    Policies = [
                        isAdminPolicy ? "RequireAdmin" : "",
                        isHRPolicy ? "HRPolicy" : "",
                        isOver18Policy ? "Over18" : ""
                    ]
                });
            }
            return View(model);
        }
    }
}
```

```

    }

    // Add Role
    [HttpPost]
    public async Task<IActionResult> AddRole(string userId, string
role)
    {
        var user = await _userManager.FindByIdAsync(userId);
        if(user != null && await
_roleManager.RoleExistsAsync(role))
            await _userManager.AddToRoleAsync(user, role);

        return RedirectToAction("ManageUsers");
    }

    // Remove Role
    [HttpPost]
    public async Task<IActionResult> RemoveRole(string userId,
string role)
    {
        var user = await _userManager.FindByIdAsync(userId);
        if (user != null)
            await _userManager.RemoveFromRoleAsync(user, role);

        return RedirectToAction("ManageUsers");
    }

    // Add Claim
    [HttpPost]
    public async Task<IActionResult> AddClaim(string userId,
string type, string value)
    {
        var user = await _userManager.FindByIdAsync(userId);
        if (user != null)
            await _userManager.AddClaimAsync(user, new
System.Security.Claims.Claim(type, value));

        return RedirectToAction("ManageUsers");
    }

    // Remove Claim
    [HttpPost]
    public async Task<IActionResult> RemoveClaim(string userId,
string type, string value)
    {
        var user = await _userManager.FindByIdAsync(userId);
        if (user != null)
            await _userManager.RemoveClaimAsync(user, new
System.Security.Claims.Claim(type, value));

        return RedirectToAction("ManageUsers");
    }

    public IActionResult Index()
    {
        return View();
    }

```

```

    }
    [Authorize] // any logged-in user
    public IActionResult UserPage()
    {
        return View();
    }

    [Authorize(Roles = "Admin")] // only Admin role
    public IActionResult AdminPage()
    {
        return View();
    }

    [AllowAnonymous] // anyone can visit
    public IActionResult PublicPage()
    {
        return View();
    }
}

public class UserViewModel
{
    public string UserId { get; set; } = null!;
    public string Email { get; set; } = null!;
    public IList<string> Roles { get; set; } = [];
    public IList<string> Claims { get; set; } = [];
    public IList<string> Policies { get; set; } = [];
}
}

```

Data/AppDbContext.cs

```

using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace WebApp7_By_Sushanka.Data
{
    public class AppDbContext : IdentityDbContext<IdentityUser>
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) :
base(options) { }
    }
}

```

Data/DbInitializer.cs

```
using Microsoft.AspNetCore.Identity;

namespace WebApp7_By_Sushanka.Data
{
    public static class DbInitializer
    {
        public static async Task Initialize(IServiceProvider
serviceProvider)
        {
            var roleManager =
serviceProvider.GetRequiredService<RoleManager<IdentityRole>>();
            var userManager =
serviceProvider.GetRequiredService<UserManager<IdentityUser>>();

            string[] roles = [ "Admin", "User" ];

            foreach (var role in roles)
            {
                if (!await roleManager.RoleExistsAsync(role))
                    await roleManager.CreateAsync(new
IdentityRole(role));
            }

            // Create default admin user
            var adminUser = await
userManager.FindByEmailAsync("user@admin.com");
            if (adminUser == null)
            {
                adminUser = new IdentityUser {
                    UserName = "user@admin.com",
                    Email = "user@admin.com",
                    EmailConfirmed = true
                };
                await userManager.CreateAsync(adminUser, "Admin@123");
            }

            // Assign role if not already in it
            if (!await userManager.IsInRoleAsync(adminUser, "Admin"))
                await userManager.AddToRoleAsync(adminUser, "Admin");
        }
    }
}
```

Views/ManageUsers.cshtml

```
<h2>User Management</h2>

<table class="table table-striped table-bordered">
  <thead class="table-dark">
    <tr>
      <th>Email</th>
      <th>Roles</th>
      <th>Claims</th>
      <th>Policies</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var user in Model)
    {
      <tr>
        <td>@user.Email</td>
        <td>@string.Join(", ", user.Roles)</td>
        <td>
          @foreach (var claim in user.Claims)
          {
            <div>@claim</div>
          }
        </td>
        <td>
          @foreach (var policy in user.Policies)
          {
            <div>@policy</div>
          }
        </td>
        <td>
          <!-- Manage Roles Button -->
          <button class="btn btn-sm btn-primary"
            data-bs-toggle="modal"
            data-bs-target="#manageRolesModal"
            data-userid="@user.UserId"
            data-email="@user.Email">
            Manage Roles
          </button>

          <!-- Manage Claims Button -->
          <button class="btn btn-sm btn-warning"
            data-bs-toggle="modal"
            data-bs-target="#manageClaimsModal"
            data-userid="@user.UserId"
            data-email="@user.Email">
            Manage Claims
          </button>

          <!-- Manage Policies Button -->
          <button class="btn btn-sm btn-success"
            data-bs-toggle="modal"
```

```

                data-bs-target="#managePoliciesModal"
                data-userid="@user.UserId"
                data-email="@user.Email">
                Manage Policies
            </button>
        </td>
    </tr>
}
</tbody>
</table>

<!-- ◇ Manage Roles Modal -->
<div class="modal fade" id="manageRolesModal" tabindex="-1" aria-
labelledby="rolesLabel" aria-hidden="true">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-primary text-white">
                <h5 class="modal-title" id="rolesLabel">Manage Roles for
<span id="rolesUserEmail"></span></h5>
                <button type="button" class="btn-close" data-bs-
dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <form method="post" asp-action="AddRole">
                    <input type="hidden" id="rolesUserId" name="userId" />
                    <div class="mb-3">
                        <label>Role Name</label>
                        <input type="text" name="role" class="form-control"
placeholder="Enter role" />
                    </div>
                    <button type="submit" class="btn btn-success">Add
Role</button>
                </form>
                <form method="post" asp-action="RemoveRole" class="mt-2">
                    <input type="hidden" id="rolesUserIdRemove"
name="userId" />
                    <div class="mb-3">
                        <label>Role Name</label>
                        <input type="text" name="role" class="form-control"
placeholder="Enter role to remove" />
                    </div>
                    <button type="submit" class="btn btn-danger">Remove
Role</button>
                </form>
            </div>
        </div>
    </div>
</div>

<!-- ◇ Manage Claims Modal -->
<div class="modal fade" id="manageClaimsModal" tabindex="-1" aria-
labelledby="claimsLabel" aria-hidden="true">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-warning">

```

```

        <h5 class="modal-title" id="claimsLabel">Manage Claims for
    <span id="claimsUserEmail"></span></h5>
        <button type="button" class="btn-close" data-bs-
dismiss="modal"></button>
    </div>
    <div class="modal-body">
        <form method="post" asp-action="AddClaim">
            <input type="hidden" id="claimsUserId" name="userId" />
            <div class="mb-3">
                <label>Claim Type</label>
                <input type="text" name="type" class="form-control"
/>

            </div>
            <div class="mb-3">
                <label>Claim Value</label>
                <input type="text" name="value" class="form-
control" />

            </div>
            <button type="submit" class="btn btn-success">Add
Claim</button>
        </form>
        <form method="post" asp-action="RemoveClaim" class="mt-2">
            <input type="hidden" id="claimsUserIdRemove"
name="userId" />

            <div class="mb-3">
                <label>Claim Type</label>
                <input type="text" name="type" class="form-control"
/>

            </div>
            <div class="mb-3">
                <label>Claim Value</label>
                <input type="text" name="value" class="form-
control" />

            </div>
            <button type="submit" class="btn btn-danger">Remove
Claim</button>
        </form>
    </div>
</div>
</div>
</div>
</div>
</div>

<!-- ◇ Manage Policies Modal (Read-only info) -->
<div class="modal fade" id="managePoliciesModal" tabindex="-1" aria-
labelledby="policiesLabel" aria-hidden="true">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header bg-success text-white">
                <h5 class="modal-title" id="policiesLabel">Policies for
    <span id="policiesUserEmail"></span></h5>
                <button type="button" class="btn-close" data-bs-
dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <p>This user's policy status is shown in the table
already.</p>

```



```

        <span>- a policy is just a logical rule that evaluates
roles/claims.</span>
    </div>
</div>
</div>
</div>
@section Scripts {
    <script>
        var rolesModal = document.getElementById('manageRolesModal');
        rolesModal.addEventListener('show.bs.modal', function (event) {
            var button = event.relatedTarget;
            var userId = button.getAttribute('data-userid');
            var email = button.getAttribute('data-email');
            document.getElementById('rolesUserId').value = userId;
            document.getElementById('rolesUserIdRemove').value = userId;
            document.getElementById('rolesUserEmail').innerText = email;
        });

        var claimsModal = document.getElementById('manageClaimsModal');
        claimsModal.addEventListener('show.bs.modal', function (event) {
            var button = event.relatedTarget;
            var userId = button.getAttribute('data-userid');
            var email = button.getAttribute('data-email');
            document.getElementById('claimsUserId').value = userId;
            document.getElementById('claimsUserIdRemove').value = userId;
            document.getElementById('claimsUserEmail').innerText = email;
        });

        var policiesModal = document.getElementById('managePoliciesModal');
        policiesModal.addEventListener('show.bs.modal', function (event) {
            var button = event.relatedTarget;
            var email = button.getAttribute('data-email');
            document.getElementById('policiesUserEmail').innerText = email;
        });
    </script>
}

```

Views/Index.cshtml

```
@{
    ViewData["Title"] = "Home Page";
}

<h1>Welcome to Auth Demo</h1>

<a href="/Home/PublicPage">Public Page</a> | <br />
<a href="/Home/UserPage">User Page (Login required)</a> | <br />
<a href="/Home/AdminPage">Admin Page (Admin only)</a> | <br />
<a href="/Home/ManageUsers">Manage Users</a>
```

Views/AdminPage.cshtml

```
<h2>Admin Page</h2>
<p>Only admins can see this.</p>
```

Views/PublicPage.cshtml

```
<h2>Public Page</h2>
<p>Anyone can see this
page.</p>
```

Views/UserPage.cshtml

```
<h2>User Page</h2>
<p>Only logged-in users can see
this.</p>
```

Program.cs

```
public class Program
{
    public static async Task Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        // Add services to the container.
        builder.Services.AddControllersWithViews();

        // 1. Add EF Core DbContext
        builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("conStr
"))));

        //builder.Services.AddDefaultIdentity<IdentityUser>(options =>
//options.SignIn.RequireConfirmedAccount = true)
//    .AddEntityFrameworkStores<AppDbContext>();

        builder.Services.AddIdentity<IdentityUser,
IdentityRole>(options =>
options.SignIn.RequireConfirmedAccount = true)
            .AddEntityFrameworkStores<AppDbContext>()
            .AddDefaultUI()
            .AddDefaultTokenProviders();
```

```

        //builder.Services.AddAuthorization(options =>
        //{
        //    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));
        //    options.AddPolicy("HR:Department", policy =>
        policy.RequireClaim("Department", "HR"));
        //    options.AddPolicy("Over18", policy =>
        policy.RequireClaim("Age", "18"));
        //});

        builder.Services.AddAuthorizationBuilder()
            .AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"))
            .AddPolicy("HR:Department", policy =>
        policy.RequireClaim("Department", "HR"))
            .AddPolicy("Over18", policy => policy.RequireClaim("Age",
        "18"));

        var app = builder.Build();

        // Configure the HTTP request pipeline.
        if (!app.Environment.IsDevelopment())
        {
            app.UseExceptionHandler("/Home/Error");
            // The default HSTS value is 30 days. You may want to
            change this for production scenarios, see https://aka.ms/aspnetcore-
            hsts.
            app.UseHsts();
        }

        app.UseHttpsRedirection();
        app.UseRouting();

        app.UseAuthorization();

        app.MapRazorPages(); // register razor pages (required for
        Identity)

        using (var scope = app.Services.CreateScope())
        {
            var services = scope.ServiceProvider;
            await DbInitializer.Initialize(services);
        }

        app.MapStaticAssets();
        app.MapControllerRoute(
            name: "default",
            pattern: "{controller=Home}/{action=Index}/{id?}")
            .WithStaticAssets();

        app.Run();
    }
}

```

Outputs:

<u>Output</u>	
WebApp7_By_Sushanka Home Privacy Register Login Welcome to Auth Demo Public Page User Page (Login required) Admin Page (Admin only) Manage Users	WebApp7_By_Sushanka Home Privacy Register Login Welcome to Auth Demo Public Page User Page (Login required) Admin Page (Admin only) Manage Users
WebApp7_By_Sushanka Home Privacy Register Login Confirm email Thank you for confirming your email. ×	

WebApp7_By_Sushanka



Home

Privacy

Register

Login

Register

Create a new account.

Email

Johndoe@gmail.com

Password

.....

Confirm Password

.....

The password and confirmation password do not match.

Register

Use another service to register.

WebApp7_By_Sushanka



Home

Privacy

Register

Login

Log in

Use a local account to log in.

Email

Johndoe@gmail.com

Password

.....

☒ Remember me?

Log in

[Forgot your password?](#)

[Register as a new user](#)

[Resend email confirmation](#)

WebApp7_By_Sushanka



Hello Johndoe@gmail.com!

Logout

Welcome to Auth Demo

[Public Page](#) |

[User Page \(Login required\)](#) |

[Admin Page \(Admin only\)](#) |

[Manage Users](#)

WebApp7_By_Sushanka

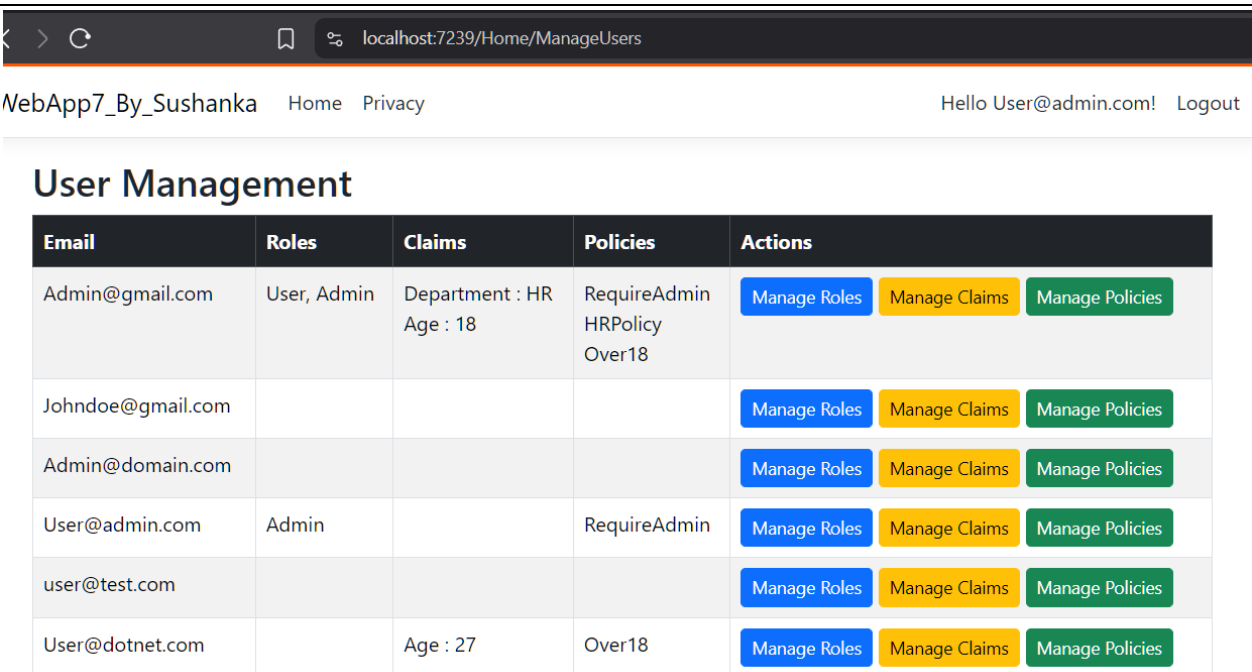
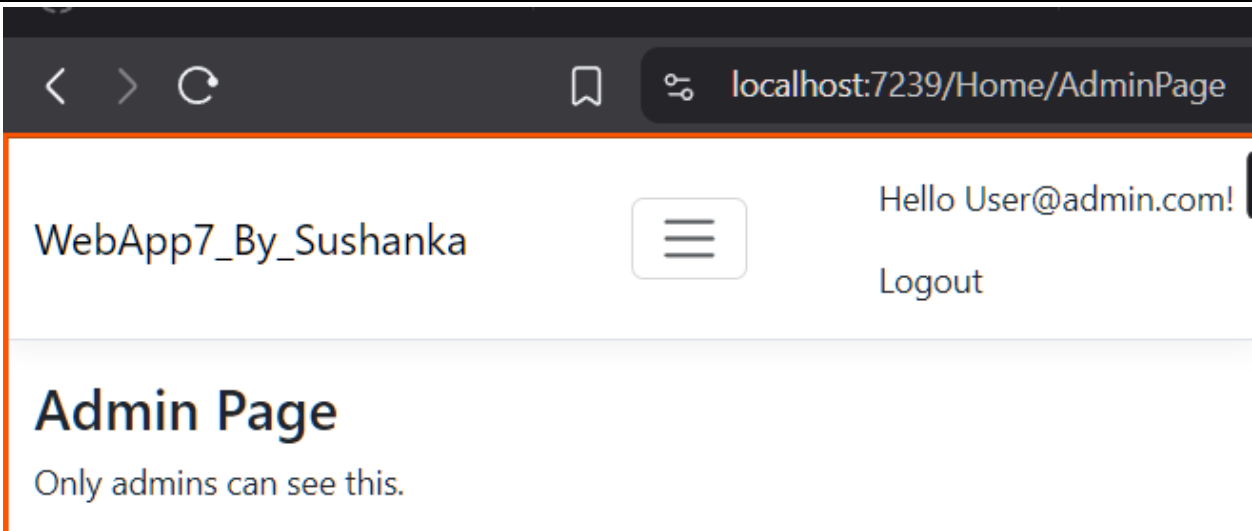
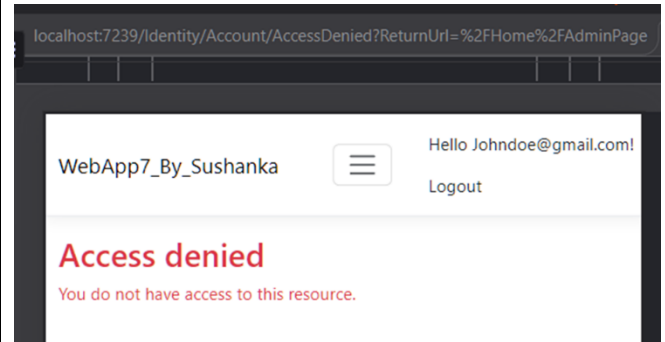
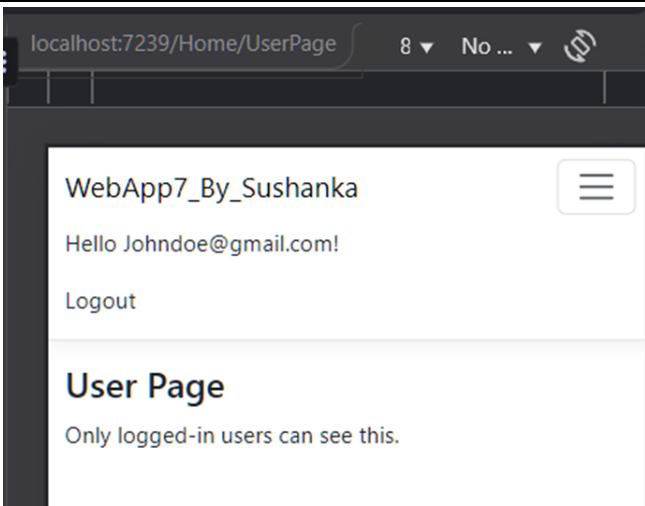


Hello Johndoe@gmail.com!

Logout

Public Page

Anyone can see this page.



Manage Roles for Johndoe@gmail.com

Role Name

User

Add Role

Role Name

Enter role to remove

Remove Role

Manage Claims for Johndoe@gmail.com

Claim Type

Department

Claim Value

HR

Add Claim

Claim Type

Claim Value

Remove Claim

Manage Claims for Admin@gmail.com

Claim Type

Claim Value

Add Claim

Claim Type

Age

Claim Value

27

Remove Claim

User Management

Email	Roles	Claims	Policies	Actions		
Admin@gmail.com	User, Admin	Department : HR	RequireAdmin HRPolicy	Manage Roles	Manage Claims	Manage Policies
Johndoe@gmail.com	User	Department : HR	HRPolicy	Manage Roles	Manage Claims	Manage Policies
Admin@domain.com				Manage Roles	Manage Claims	Manage Policies
User@admin.com	Admin		RequireAdmin	Manage Roles	Manage Claims	Manage Policies
user@test.com				Manage Roles	Manage Claims	Manage Policies
User@dotnet.com		Age : 27	Over18	Manage Roles	Manage Claims	Manage Policies

28. Hosting & Deployment

On monsterasp.net

Login and create new website

Enter subdomain name

admin.monsterasp.net/app/site/add/step2

MonsterASP.net
ASP.NET & .NET Cloud Hosting

Create new service

Tasks 0

Messe

Websites

Dashboard
Account overview

Websites
Cloud websites 0

+ Add website
Create new

Databases
Cloud databases 1

E-mails
Cloud mails 0

DNS
DNS management 0

Tickets
Support tickets 0

Subscriptions
Packages 0

Billing
Billing management

Customer
Account management

Create website

Here you can create new website

Plan Domain Subscription Location

Hosting plan Domain and type Package Datacenter

FREE Subdomain
Website on our Free subdomain

Your Domain
Website on your own domain

Your Subdomain
Website on your own subdomain

Free Subdomain
Create website with our Free subdomain

Subdomain name Free domain

sushanka .runasp.net

Type subdomain name like: [mypersonalsite](#)

Min. 3 characters. Only letters [a-z] numbers [0-9] and dash [-] are allowed.

INFO Website will be created on our Free subdomain. After creation will be immediately available on your chosen subdomain name.

TIP Only for **Premium plans** you can add your own custom domain or subdomain like: [yourdomain.com](#) etc.

sushanka.runasp.net
Website details [sushanka.runasp.net](#)

Show Website

Overview

Site settings and details

Free plan

ID site35621

Status Enabled

Domain [sushanka.runasp.net](#)

Name [sushanka.runasp.net](#) [edit / rename](#)

Plan FreeSite

Subscription [open](#)

Server [site35621.siteasp.net](#) [EU] [W7]
Always use this server name instead of IP address.

.NET .NET 10.x/9.x/8.x/7.x/6.x/5.x/Core [x86] [edit](#)

PHP [NONE] [edit](#)

Memory limit 256 MB RAM

File count 0x 10000x

Quota usage 0 MB 5120 MB

[Recalculate](#)

Disk quota

0 MB 5120 MB

Access to website

FTP and SFTP login details

Server / Hostname [site35621.siteasp.net](#) [EU] [W7]
Always use this server name instead of IP address.

Port 21 [FTP]
22 [SFTP/SCP]

Login site35621

Password ***** [edit](#) [Show password](#)

Website Root /wwwroot

WebFTP [https://webftp.monsterasp.net](#) [open](#)
This link will automatically sign you in to web interface.

NOTE With these login details you can access website via FTP/SFTP and upload your content directly to website. We recommend using [Filezilla client](#) for FTP/SFTP access. Upload all your application files to existing /wwwroot directory.

HELP [How to connect to FTP/SFTP from FileZilla client?](#)

Your website is ready...

[Reload](#)

DNS status

Current DNS pointing status

Domain [sushanka.runasp.net](#)

Type Free subdomain

NS server [Our servers \[NS\]](#)

Web server [Our server \[A\]](#)

Domain [sushanka.runasp.net](#) pointing correctly to our web server 178.63.129.220

[Refresh](#)

Deploying using FTP/WebDeploy

Disk quota

0%
0 MB 5120 MB

BACK to website list

Files File manager

Domains Aliases / Bindings

HTTPS SSL certificates

Deploy FTP / WebDeploy

Scripting ASP.NET / PHP

Application installer

Process list

Tasks Planned jobs

Resources Resource usage

Logs Error logs

Upgrade Transfer / ScaleUp

WebFTP Interface

Show Website

Site usage
Weekly site traffic overview

Status publishing correctly to our web site
178.63.129.220
Refresh

Quick actions
Restart Stop

Quick statistics
Site uptime 0min
AppPool uptime 0min
Recycled 0
Crash 0
GET 0
HEAD 0
POST 0
404 0
Transfer 0 MB

Now, download publish profile

Status	Login	Path	Password	Permission	Last login
Enabled	site35621 [\]	\	***** edit	Read+Write edit	04.09. 2025 20:15

FTP/SFTP Access
FTP and SFTP login details

FTP Server site35621.siteasp.net [EU]
Always use this server name instead of IP address.
Port 21 [FTP]
22 [SFTP/SCP]
Login site35621
Password ***** edit Show password
Website Root \wwwroot
WebFTP https://webftp.monsterasp.net open
This link will automatically sign you in to web Interface.
NOTE With these login details you can access website via FTP/SFTP and upload your content directly to website. We recommend using Filezilla client for FTP/SFTP access. Upload all your application files to existing /wwwroot directory.
HELP How to connect to FTP/SFTP from FileZilla client?

WebDeploy Access
Visual Studio login details

Enabled Disabled
Server site35621.siteasp.net [EU]
Always use this server name instead of IP address.
Port 8172
Site site35621
Login site35621
Password ***** edit Show password
Download publish profile
NOTE WebDeploy is best way to deploy your applications directly from Visual Studio. Simply download publish profile and import it into your project in Visual Studio to get started.
HELP How to publish website using WebDeploy from Visual Studio?

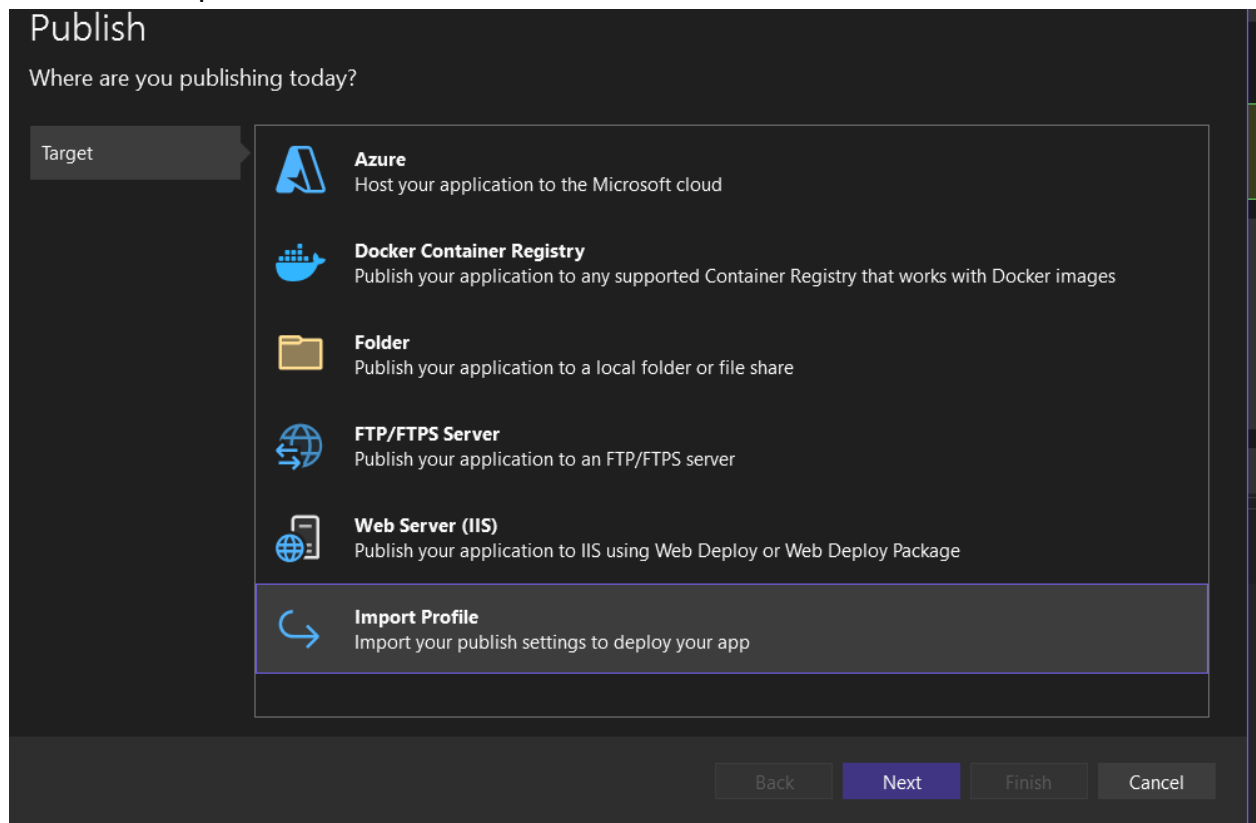
Pictures Last month

File name: sushanka.runasp.net-WebDeploy.publishSettings

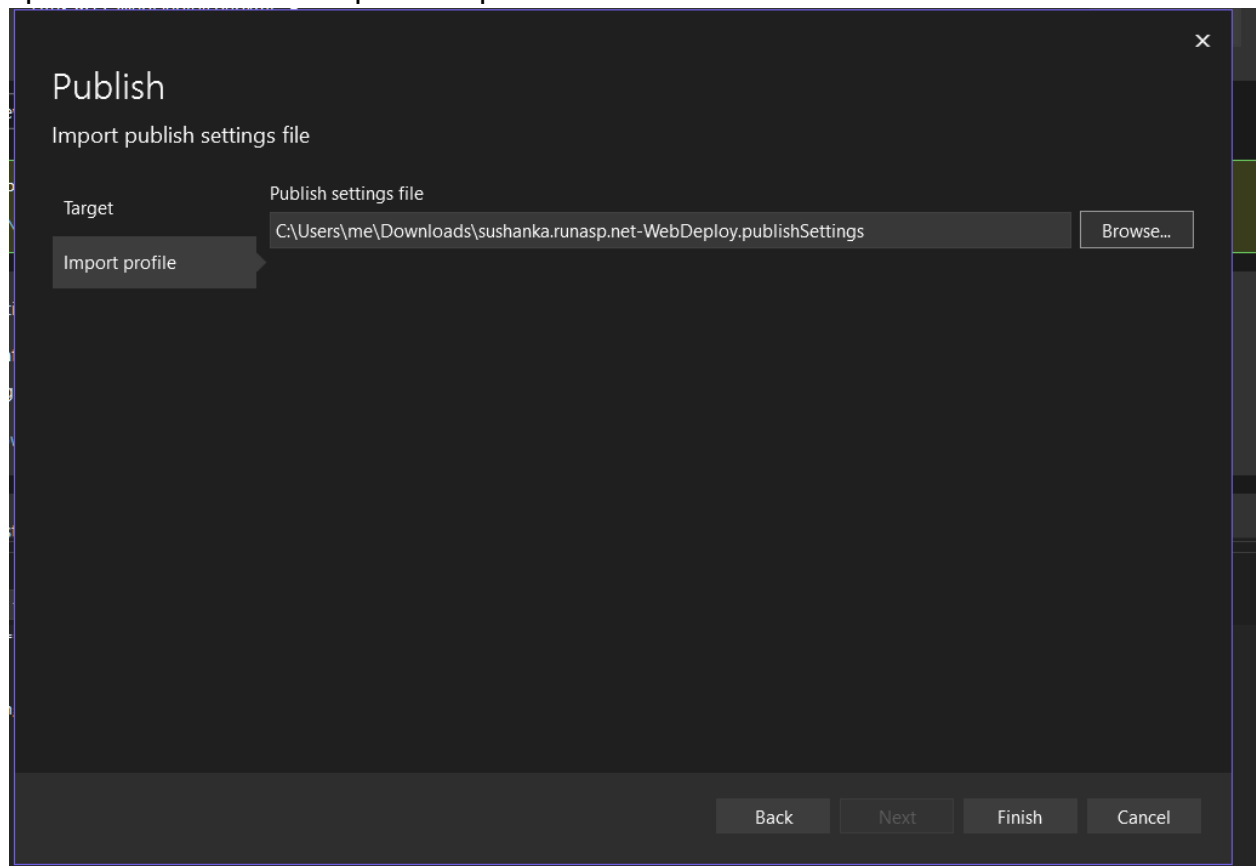
Save as type: PUBLISHSETTINGS File (*.publishSettings)

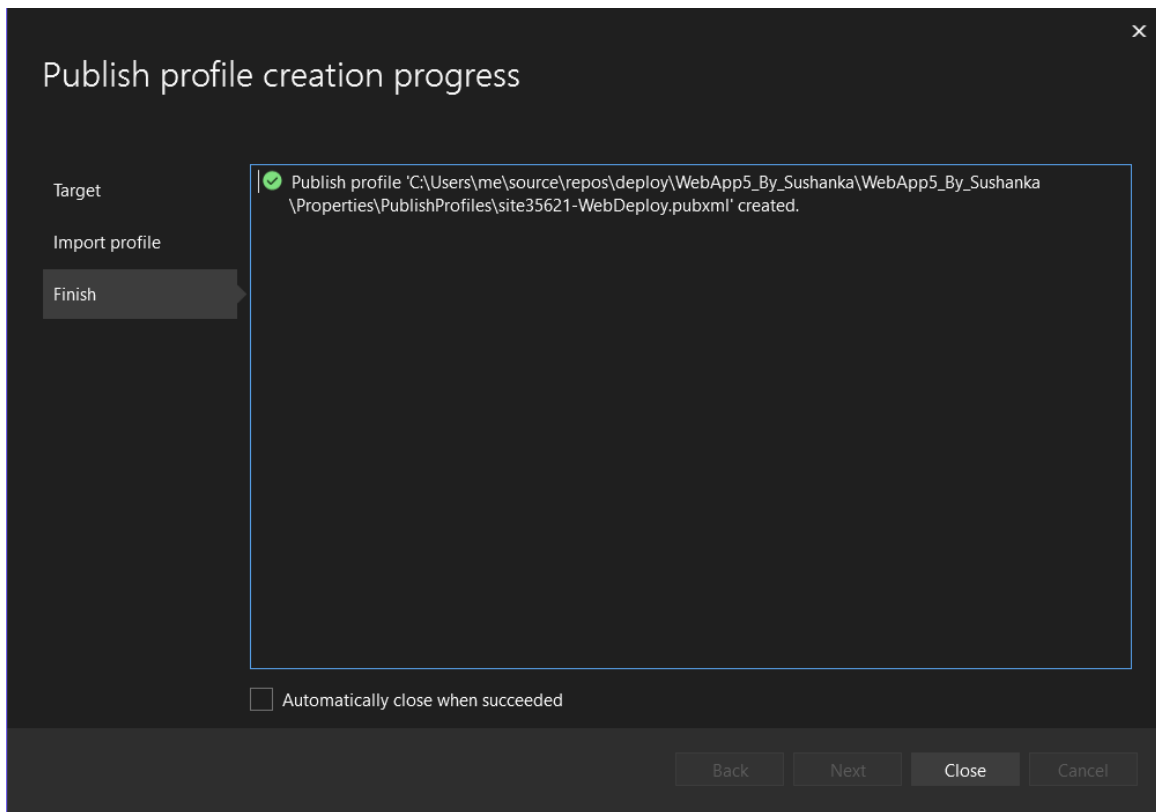
Hide Folders Save Cancel

Now Publish Dot Net Project
Select as Import Profile

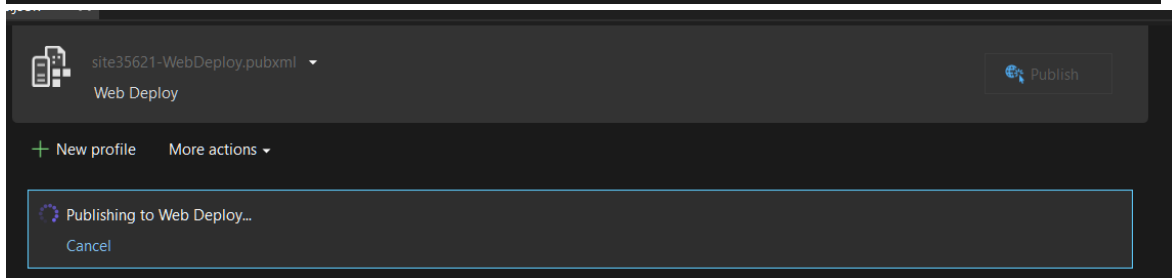
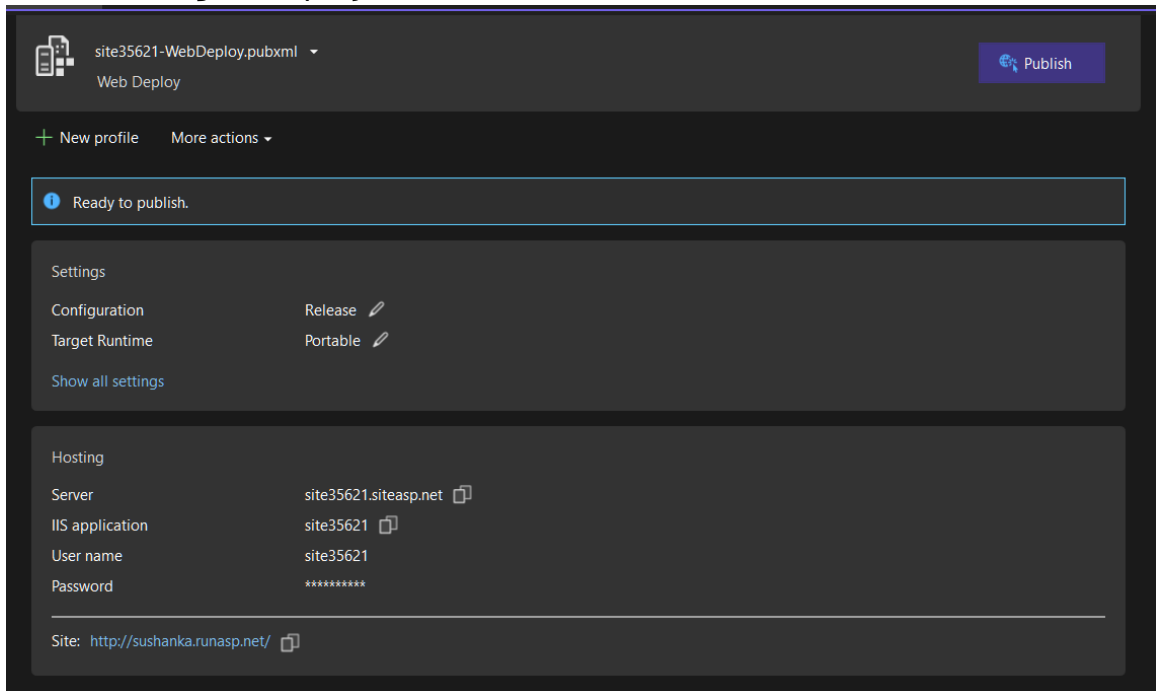


Upload the downloaded publish profile

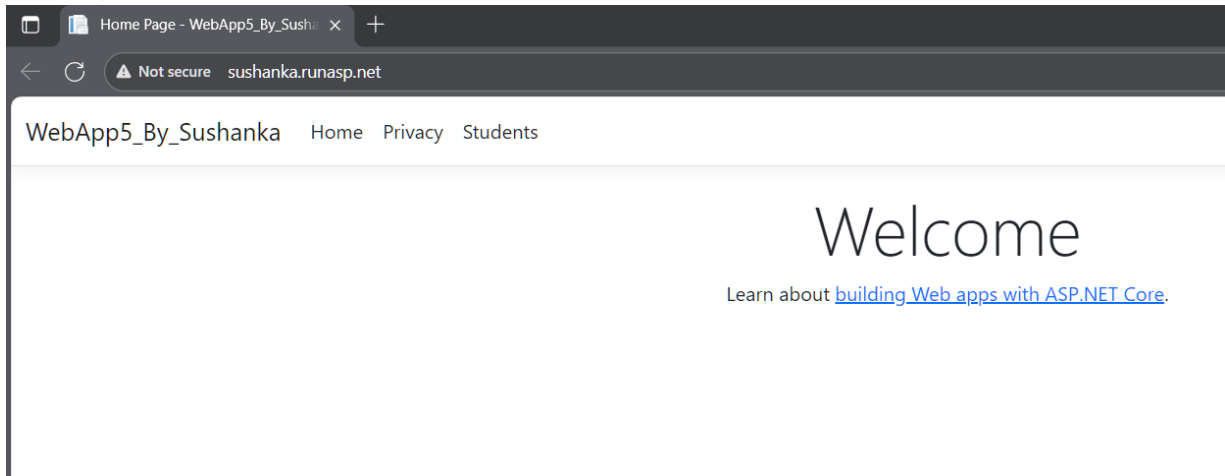




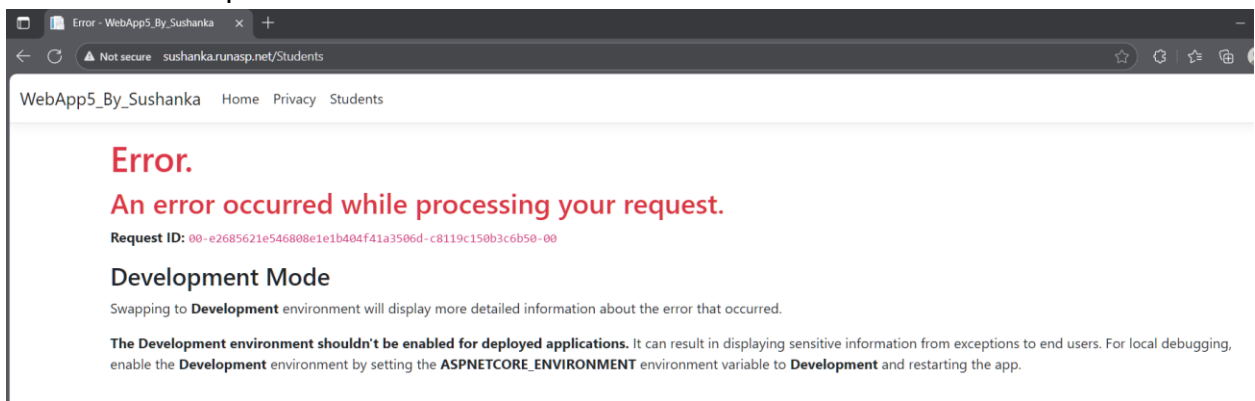
Publish using WebDeploy



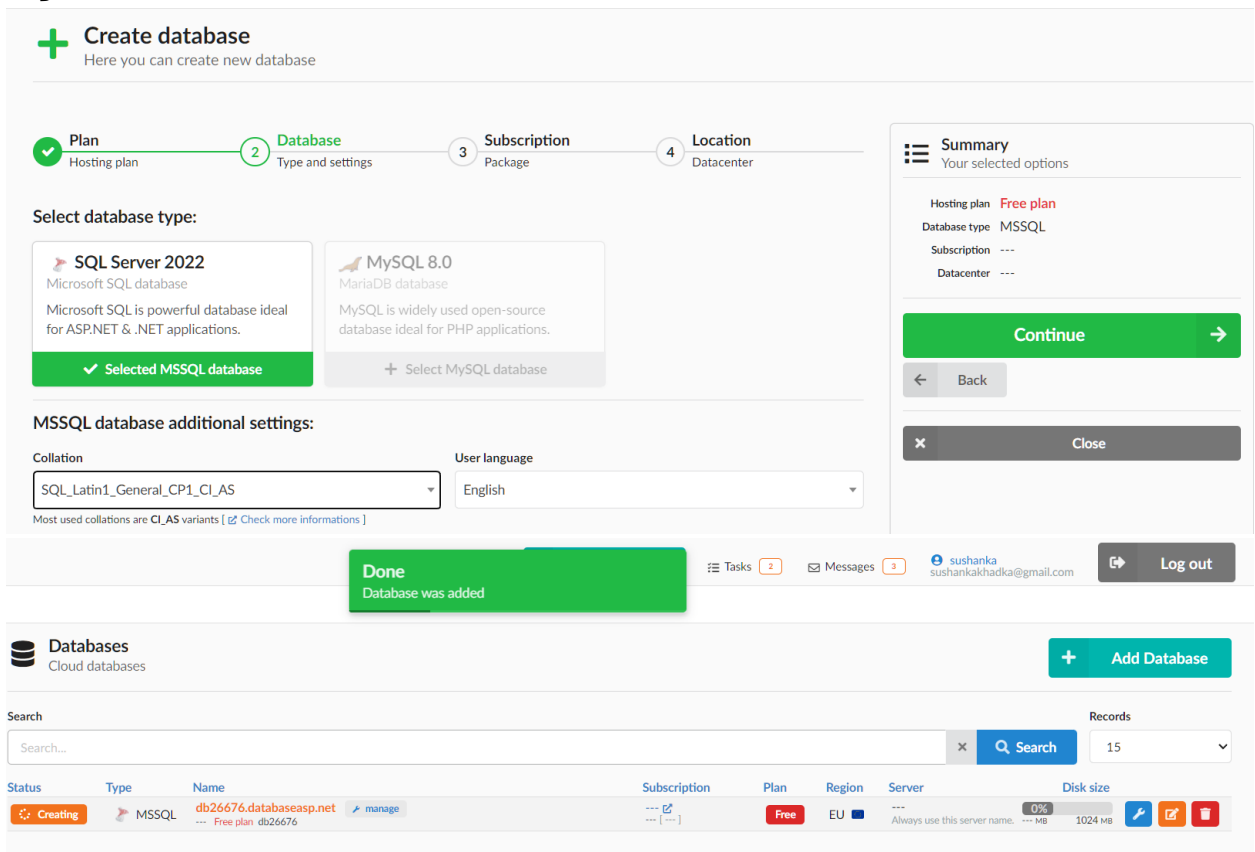
Visit your website



Check each component



Might be database error let's create one.



Copy the connection string

db26676.databaseasp.net
Database overview db26676.databaseasp.net [MSSQL]

Overview
Database settings and details

ID db26676
Status Enabled
Name db26676.databaseasp.net
Database db26676.databaseasp.net
Type MSSQL
Plan FreeDatabase
Subscription
Server db26676.databaseasp.net [EU] [SS]
Collation SQL_Latin1_General_CP1_CI_AS
Compatibility SqlServer2022
Quota usage 1% 8 MB 1024 MB
Disk quota 1% 8 MB 1024 MB

Local access [for Websites]
MSSQL local access login details

Server / Hostname db26676.databaseasp.net [EU] [SS]
Port 1433 [MSSQL]
Login db26676
Password *****
WebMSSQL https://webmssql.monsterasp.net

Connection string
Server=db26676.databaseasp.net; Database=db26676; User Id=db26676; Password=*****; Encrypt=False; MultipleActiveResultSets=True;

Go to WebFtp

webftp.monsterasp.net/mftp/

wwwroot

zh-CN

zh-Hans

zh-Hant

appsettings.Development.json

appsettings.json

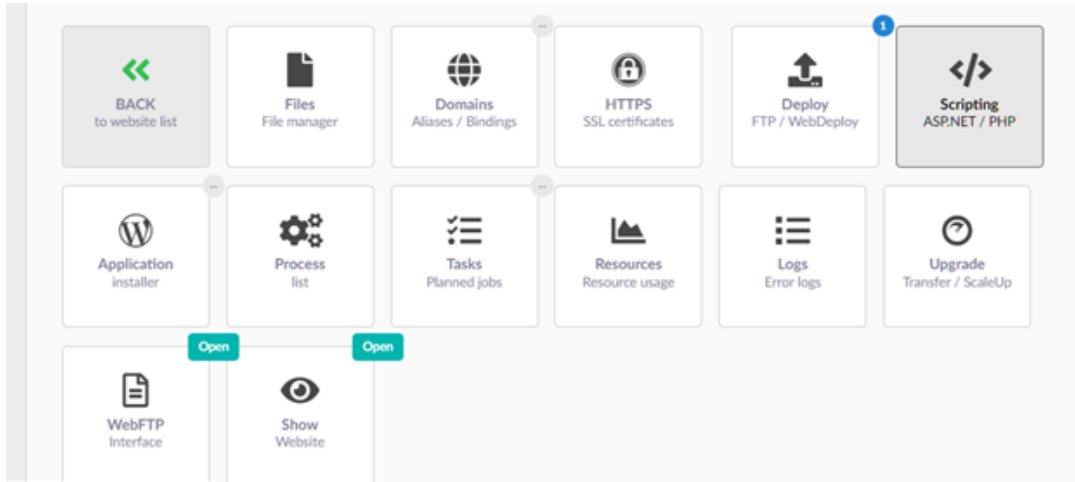
Open wwwroot/appsettings.json

```
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*",
9   "ConnectionStrings": {
10    "conStr": "Server=sql.bs.site.net\\MSSQL2016; Database=sushankakhadka_WebApp5_DbFirst; User Id=sushankakhadka; Password=admin123; TrustServerCertificate=True;"
11  }
12 }
13
```

Now paste the ConnectionString copied earlier

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "conStr": "Server=db26676.databaseasp.net; Database=db26676; User Id=db26676; Password= Z=j9g4-E5Pt_; TrustServerCertificate=True;"
  }
}
```

Reload the site if still not working switch to development environment
Goto Scripting.



Navigate to Environment variables

ASP.NET / .NET

ASP.NET / .NET settings

Version

.NET 10.x/9.x/8.x/7.x/6.x/5.x/Core

▼

Save

Current version: .NET 10.x/9.x/8.x/7.x/6.x/5.x/Core

Mode

Integrated [IIS 10.x/8.x/7.x] [Recommended]

▼

Save

Current mode: Integrated

Bitness

32-bit [x86] [Low RAM consumption] [Recommended]

▼

Save

Current bitness: x32bit

NOTE

For best performance and lower memory consumption of your application we recommended using 32-bit [x86]. Use 64-bit [x64] only if you application really requires it.

Advanced settings

Environment variables

AspNetCore

AspNetCore settings

Startup process

dotnet .\WebApp5_By_Sushanka.dll

Path to application startup process with arguments.

Hosting model

InProcess [Recommended]

▼

Current hosting model: inprocess

WARNING

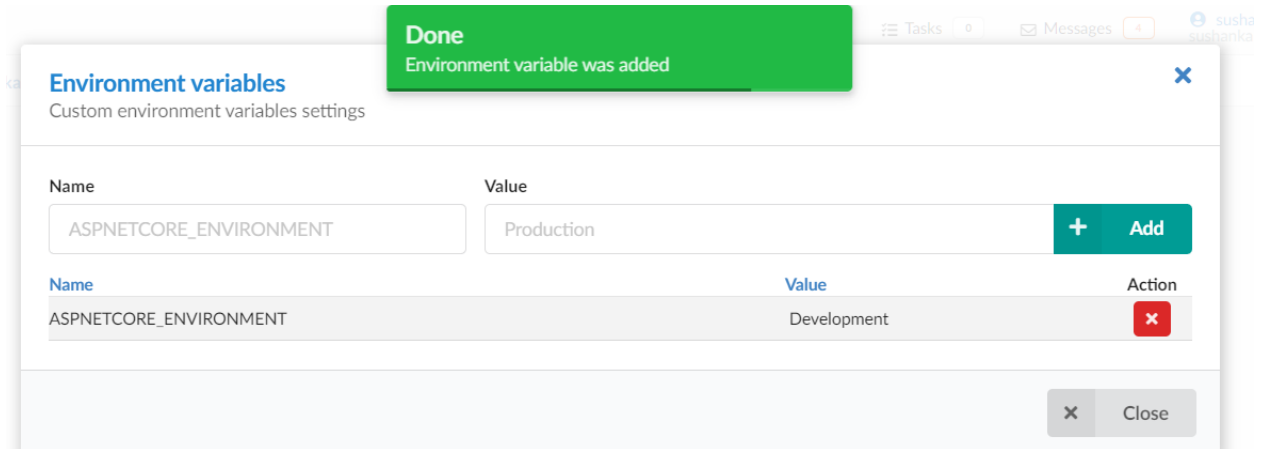
Hosting model OutOfProcess with custom EXE runtime is not supported! See our help for more information: [ASP.NET Core hosting model support?](#)

HELP

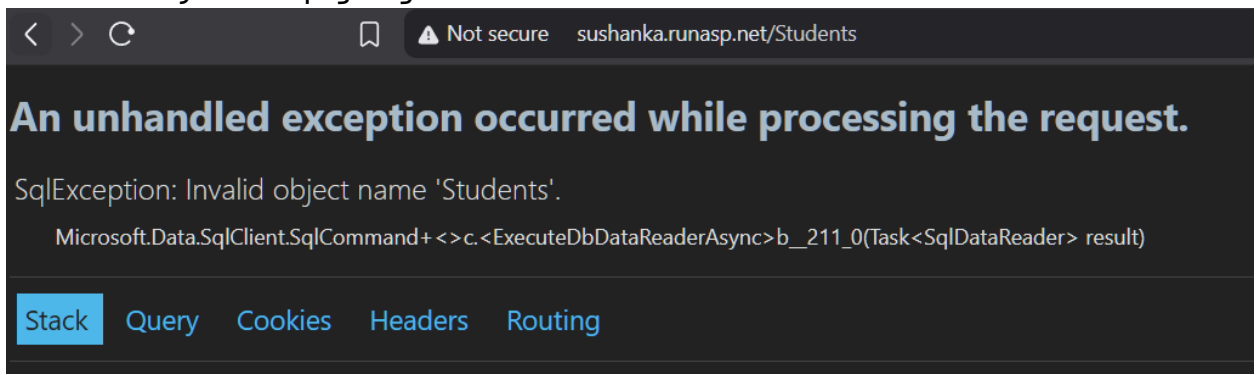
[Which ASP.NET Core hosting models are supported?](#)

Save settings

Add environment variables as:

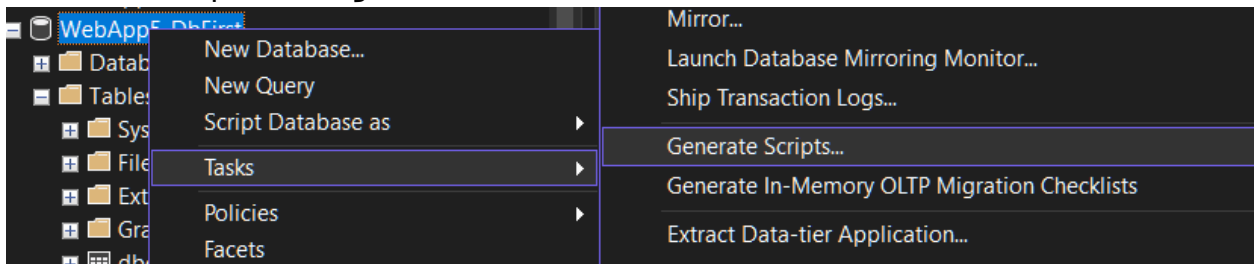


Reload the your webpage again

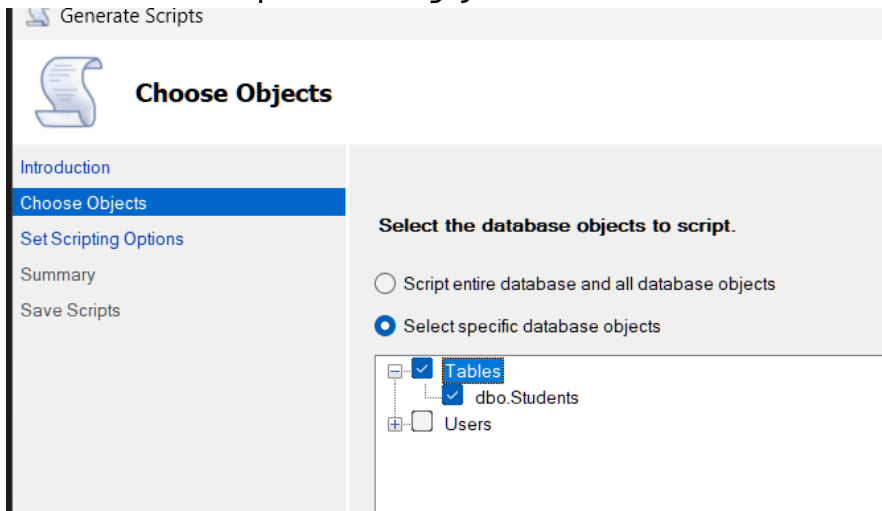


Now its clear more clear: the server is connected with database correctly but our database don't have table/object named 'Students'

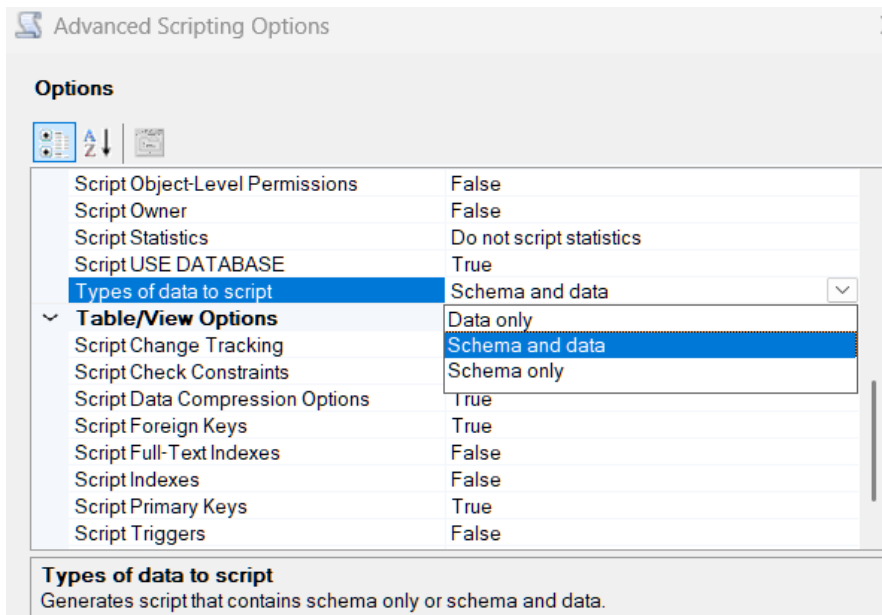
Open Sql Server Management Software (SSMS) and find the database. Find Generate Scripts on Right click to that database as follows.



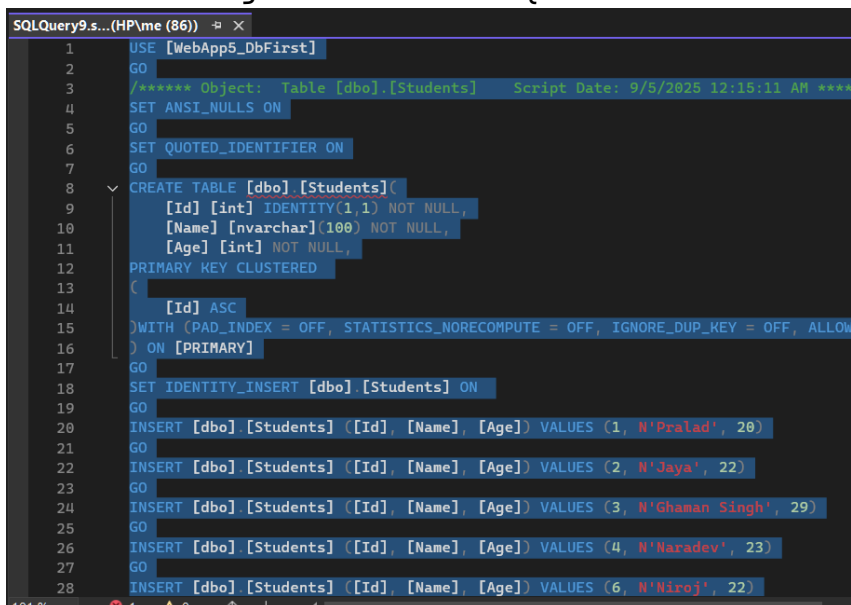
Follow these steps accordingly.



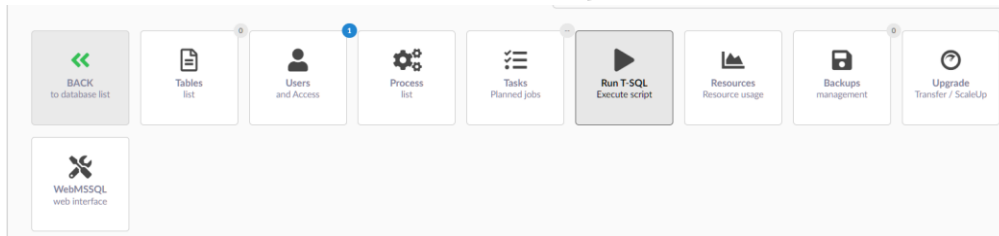
Click on Advanced tab and select Schema and data



This will auto generate the raw SQL from the database table.



Visit Run Sql section on the admin page.



Copy paste the sql queries and update the database name.

**db26676.databaseasp.net**
Run T-SQL script [db26676.databaseasp.net](#) [MSSQL]

**Run T-SQL**
Enter your T-SQL batch script. It will be executed on database: [db26676.databaseasp.net](#)

```
USE [db26676]
GO
/***** Object: Table [dbo].[Students]  Script Date: 9/5/2025 12:15:11 AM *****/
SET ANSI_NULLS ON
GO
SET QUOTED_IDENTIFIER ON
GO
CREATE TABLE [dbo].[Students](
    [Id] [int] IDENTITY(1,1) NOT NULL,
    [Name] [nvarchar](100) NOT NULL,
    [Age] [int] NOT NULL,
    PRIMARY KEY CLUSTERED
(
    [Id] ASC
)WITH (PAD_INDEX = OFF, STATISTICS_NORECOMPUTE = OFF, IGNORE_DUP_KEY = OFF, ALLOW_ROW_LOCKS = ON, ALLOW_PAGE_LOCKS = ON) ON [PRIMARY]
```

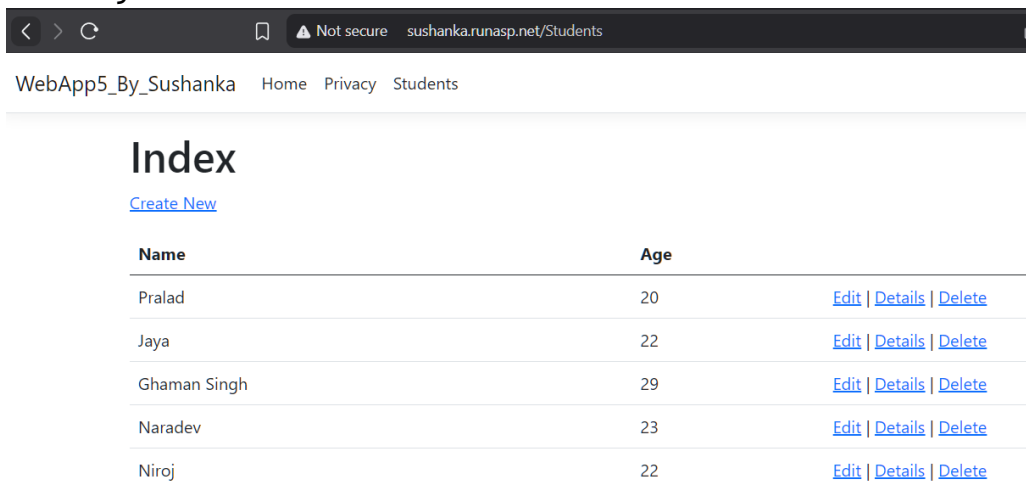
☒ **Close all connections?**
This action close all connections to database before execute T-SQL script.



Result:

DONE: SQL script was executed in database

Reload your site



Now everything will be functional.

Edit

Student

Name

Niroj

Age

22

Save

[Back to List](#)

Edit

Student

Name

Sushanka

Age

22

Save

[Back to List](#)

Index

[Create New](#)

Name	Age	
Pralad	20	Edit Details Delete
Jaya	22	Edit Details Delete
Ghaman Singh	29	Edit Details Delete
Naradev	23	Edit Details Delete
Sushanka	22	Edit Details Delete

< > ↺  ⚠ Not secure sushanka.runasp.net/Students/Delete/6

WebApp5_By_Sushanka Home Privacy Students

Delete

Are you sure you want to delete this?

Student

Name Sushanka

Age 22

Delete

| [Back to List](#)

***** The End *****